



CO3202 Entrepreneurial Project

Dissertation

“LearnCade: A platform for transforming classic arcade games into engaging, educational experiences.”

Rahul Rathod

April 2026

School of Computing and Mathematical Sciences, University of Leicester

Abstract

There are many educational technology platforms available that offer interactive learning experiences for students, however the majority are limited to simple quiz formats with little to no real gameplay depth. This project follows the development of LearnCade, a web-based platform that transforms classic arcade games into engaging, personalised educational experiences for children aged 5 to 10. LearnCade allows users to create fully customised educational games by selecting a game template, educational topic and game parameters. The platform also includes unique features such as Classroom Mode, giving teachers the ability to host live game sessions for an entire class of students in real time.

To realise this project, a range of technical and entrepreneurial skills were applied, including software engineering, game development, UI/UX design, prompt engineering and business development. This dissertation covers the full development of LearnCade, from the initial aims and objectives, the business model, design through to implementation and evaluation. Overall, a successful platform has been built that meets all its objectives and has strong plans for future development.

Contents

Abstract	2
1. Introduction	5
1.1 Aims	5
1.2 Objectives.....	5
1.3 Challenges.....	7
2. Survey of Literature	7
2.1 Background Research.....	7
2.2 Competitor Research	8
2.2.1 Competitor Websites	8
2.2.2 Competitor Traffic Analysis.....	10
3. Business Development	10
3.1 Unique Value Proposition (UVP)	10
3.2 Target Audience.....	10
3.3 Customers	12
3.4 Lean Canvas	12
3.5 The Market	12
3.5.1 Market Size and Research.....	12
3.5.2 Ways to market	13
3.5.3 Competition	13
3.5.4 Strategic Partnerships	16
3.6 Revenue Model.....	16
3.6.1 Monetisation.....	16
3.6.2 Revenue and Expenses.....	17
4. Design	19
4.1 Design Software	19
4.2 User Flow	19
4.3 Logo Design	20
4.4 Wireframes	21
4.5 Game Design.....	25
4.6 Usability Considerations.....	28
5. Implementation	29
5.1 System Architecture	29
5.2 API Integration.....	30
5.3 Games	36

5.4 Platform Features	48
5.5 Database	54
5.6 Landing Page.....	56
5.7 Play Page.....	58
5.8 Error Handling	60
5.9 Deployment	61
6. Results and Discussion	61
6.1 Aims Evaluation.....	61
6.2 Objectives Evaluation	62
6.3 System Testing	64
7. Critical Appraisal	66
7.1 What Went Well.....	66
7.2 What Didn't Go As Well.....	66
7.3 Social, Commercial and Economic Considerations	67
7.4 Personal Development	67
8. Conclusion	67
9. References.....	68

1. Introduction

Children today grow up immersed in digital experiences through phones, tablets and computers. Because of this, traditional educational tools and teaching methods are starting to feel outdated as they fail to engage the new generation of students. This is where my project, LearnCade, comes in as a forward-thinking solution.

LearnCade is a web-based platform that allows users to create their own 2D educational games that are designed to engage and educate children. The application involves the user selecting a game template, an educational topic, and a set of details to create a fully customised experience. To differentiate itself from traditional learning sites, LearnCade automatically generates personalised content combined with engaging arcade gameplay to provide a truly unique learning experience. Additionally, users can share and play these custom games in a classroom environment, giving LearnCade the potential to be used in schools.

1.1 Aims

The primary aims for the project are the following:

- Provide a platform that balances both education and entertainment by effectively teaching through games. I want to take a gameplay focused approach, meaning users engage with the gameplay first, and then answering the questions and completing levels is part of the experience rather than feeling like a test.
- To deliver a strong arcade experience by basing the games on popular classic titles and designing the UI with arcade elements such as pixel fonts, dark backgrounds, retro music and sound effects.
- To transform the genre of play-based learning through automatic generation of instant, personalised experiences, allowing for immediate adaptability to different subjects, age groups and preferences.
- To support classroom learning, through a Classroom Mode that allows teachers to host sessions, invite students, and create friendly competition.

Additional aims include game sharing and account creation with a profile where users can view their scores, recent games, and replay previous games.

1.2 Objectives

In order to achieve these aims, the following objectives will need to be met:

Objective 1 – Game Creation Flow

LearnCade should allow users to create a fully customised educational game by selecting a game template, educational topic and game parameters such as lives, questions and a timer. Each topic will offer additional options, Mathematics will offer operations such as Addition, Subtraction, Multiplication and Division, Spelling will offer word categories such as Animals and Countries, and Science will offer categories such as Plants, Space and the Human Body. An optional custom theme will also allow users to enter colours to customise the game visually.

Objective 2 – Develop Arcade Games

LearnCade will feature three fully functional arcade games adapted from classic titles. Maze Runner inspired by Pac-Man, Space Shooter inspired by Space Invaders, and Brick Breaker inspired by Breakout. 2D game assets such as character sprites, enemies and power-ups will be designed using Krita, ensuring a unique visual identity that matches the arcade aesthetic. Each game should be playable from start to finish, include a scoring and multiplier system, power-ups and seamless integration with parametrised generation of personalised educational content.

Objective 3 – Integrate the API

The Claude API will be integrated to dynamically generate educational questions, answers and custom themes based on the user's selections, removing the need to manually program thousands of potential outcomes.

Objective 4 – Strong UI/UX Design

LearnCade will feature a polished user interface with a clear landing page, intuitive navigation, consistent styling and appropriate error handling. The dual audience presents a key design challenge as the interface must feel engaging for children without appearing overly childish to teachers or possibly parents.

Objective 5 – Create Classroom Mode

LearnCade will include a Classroom Mode allowing teachers to host live game sessions with an intuitive host, join flow and session code system, making it easy to setup and share. This feature will also include a real time leaderboard, creating fun competition in a classroom environment.

Objective 6 – Enable Game Sharing

Users should be able to share their created games through a unique share code, allowing others to play identical game experiences across different devices.

Objective 7 – Account Profile functionality

LearnCade will include a user account system where users can create an account, log in and access a profile displaying key stats such as overall score, high score, alongside the ability to view and replay recently created games.

Objective 8 – Database Integration

A reliable database will be required to store account information, all created games and real-time session data for Classroom Mode.

1.3 Challenges

Several challenges during the development of LearnCade have been identified below:

1. Integrating the Claude API to generate a structured dataset of questions, answers and custom themes required strong prompt engineering to ensure correct, consistent, and age-appropriate outputs. This is discussed in Section 5.2.
2. Developing three fully functional arcade games that were engaging and educational while maintaining the arcade aesthetic presented varied design and technical challenges, discussed in Section 4.5 and Section 5.3.
3. Building Classroom Mode presented challenges around real-time data, requiring a strong backend solution capable of handling multiple connections and live leaderboard updates. This is discussed in Section 5.4 and Section 5.5.
4. A significant design challenge was creating a platform that appeals to both children and teachers. Addressing this required careful wireframing and usability considerations, as discussed in Section 4.4 and Section 4.6.

2. Survey of Literature

The following chapter summarises the research undertaken for LearnCade.

2.1 Background Research

Educational Games Research:

A key motivation behind building LearnCade was the positive research supporting the effectiveness of educational games as a learning tool. The research paper by Pena-Miguel and Sedano Hoyuelos describes educational games as "highly motivating" tools that "communicate very efficiently the concepts and facts of many subjects" [1], directly reflecting the core goal of LearnCade. Their research also highlights that successful educational games are designed to "balance the subject matter with the gameplay" [1], which strongly aligns with LearnCade's gameplay focused approach.

The paper also presents measurable real-world results, 85% of users passed exams after using educational games, and 82.35% of parents reported them as valuable learning tools [1], supporting the decision to target parents as discussed later in Section 3.3. Additionally, the paper emphasises that educational games are most effective when they include a structured element that guides the learner through the experience [1], which directly aligns with LearnCade's game creation flow, as mentioned in Section 1.2.

Technical Research:

Undertaking a project of this scope required careful research into the technologies best suited to the platform before development began. The following section outlines the key decisions made behind my choices.

Next.js was selected as the core framework as it provides built-in API routes, allowing API calls to be handled server-side, and offers seamless integration with Vercel for deployment [2]. Next.js also has well-documented official tutorials and courses available for developers [3].

TypeScript was chosen as the primary programming language due to its static typing system [4], meaning type errors are caught in the editor before runtime. For a project of this complexity, this made TypeScript a significantly more reliable choice over JavaScript.

The Claude API was selected as informal testing demonstrated that Claude produced high-quality and consistent outputs when prompted to generate game-related or educational content, whereas other LLMs such as ChatGPT and Gemini occasionally lacked creativity or understanding [5].

Supabase [6] was chosen as the database solution due to its built-in real-time capabilities, which particularly suited the live leaderboard in Classroom Mode, and its intuitive dashboard for managing user data and scores.

2.2 Competitor Research

The following section analyses the key competitors that I researched before development:

2.2.1 Competitor Websites

As part of the research process, a range of existing play-based learning platforms were explored to identify opportunities for LearnCade to differentiate itself.

The first platform analysed was Educaplay [7], one of the most similar platforms to LearnCade, using templates and an LLM to create educational activities such as Crosswords, Quizzes and Word Searches.

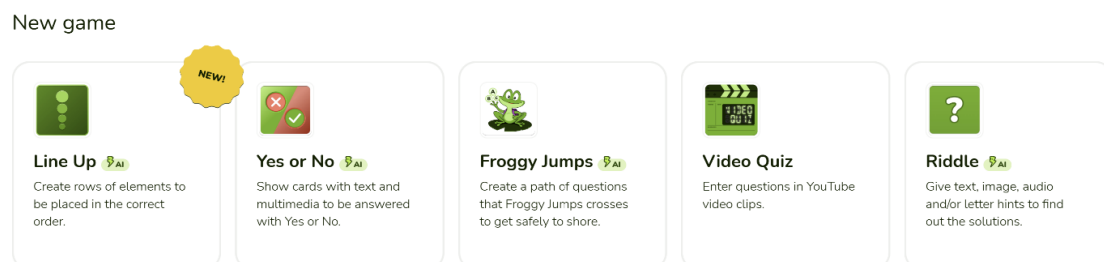


Figure 2.1 – Educaplay, New Game screen

Figure 2.1 shows the ‘New Game’ screen within the Educaplay platform which is a page that presents several learning activity templates. The overall design feels organised and approachable.

Furthermore, Educaplay’s creation flow includes users directly typing what they would like to create, enabling users to be creative and have unlimited options. However, it can also cause issues as users may give irrelevant or insufficient information leading to a low quality learning experience.

Figure 2.2 shows a generated activity using the prompt "5 times table game" and Froggy Jumps template. Whilst the platform generates relevant questions, the gameplay is minimal, simply moving a frog between lily pads with no real mechanics. This highlights the key gap that LearnCade aims to fill.



Figure 2.2 – Educaplay, Froggy Jumps

The second competitor analysed was Kahoot! [8]. Released in 2013, Kahoot! has achieved over 10 billion participants [9], demonstrating the strong demand for interactive learning platforms in classroom settings.

Initially, the Kahoot! Generator uses an LLM to generate quiz content as shown in Figure 2.3, however like Educaplay, the experience lacks meaningful gameplay as Kahoot! only offers simple quiz slides.

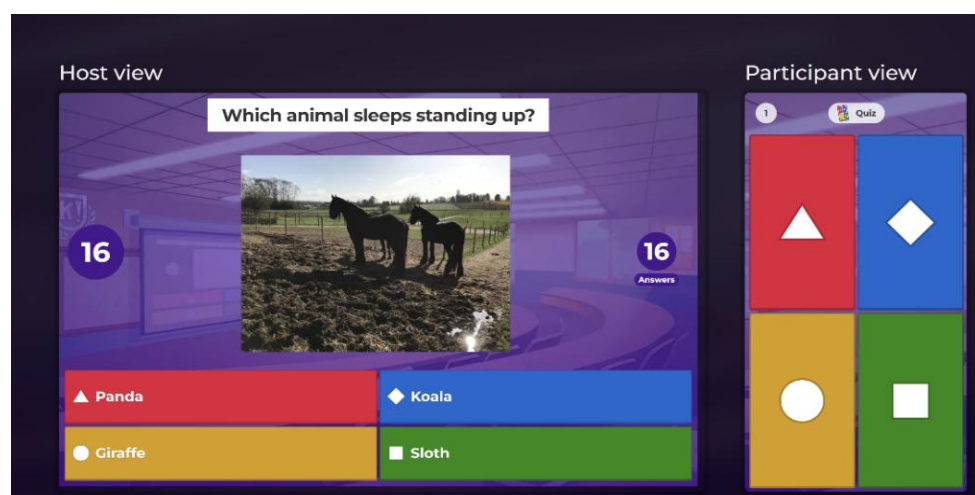


Figure 2.3 – Kahoot! Generator output

More significantly, Kahoot!'s multiplayer session system was a key source of inspiration for LearnCade. Teachers host a session and share a unique PIN with their class, enabling all students to join and compete simultaneously. This directly informed the development of LearnCade's Classroom Mode.

2.2.2 Competitor Traffic Analysis

In order to explore my competitors further, I used SimilarWeb [10] to analyse web traffic for both platforms discussed in Section 2.2.1.

	Educaplay	Kahoot
Average monthly visitors (Oct 2025)	3.4 M	68.7 M
Visit duration	2 mins 47 sec	8 mins 54 sec
Bounce rate	52.9%	26.6%
Country with largest traffic share	Mexico (18.4%)	United States (39.2%)

Figure 2.4 – Competitor web traffic analysis (created in the interim report, data is from October 2025)

Figure 2.4 presents the findings of the web traffic analysis. Kahoot's 68.7 million monthly visitors and 8 minute 54 second average visit duration demonstrates the strong demand for classroom learning platforms, likely driven by its teacher-led session format.

Educaplay's shorter visit duration of 2 minutes 47 seconds highlights an opportunity for LearnCade to differentiate through more engaging gameplay, rather than simple games with no real game mechanics.

Kahoot's low bounce rate of 26.6% compared to Educaplay's 52.9% reinforced the importance of strong UX/UI design, particularly in the landing page.

Finally, with Kahoot's largest traffic share coming from the United States and Educaplay's from Mexico, neither platform is primarily dominant in the UK, presenting a clear opportunity for LearnCade as a platform designed for UK primary school education.

3. Business Development

The following chapter outlines the process that I have adopted in developing LearnCade's business idea:

3.1 Unique Value Proposition (UVP)

"Where Learning Levels Up!"

This catchphrase effectively captures the core vision of LearnCade. The phrase works on two levels, "learning" speaks directly to the educational purpose, whilst "levels up" references the gaming side and represents progression and achievement. The UVP appeals to both audiences, for children "levels up" is exciting, while "learning" reassures teachers and parents of its educational value.

3.2 Target Audience

LearnCade primarily targets children aged 5 to 10, as the educational topics implemented within the platform are most suitable for this age range.

As of 2025, there are over 4.5 million pupils enrolled in state-funded primary schools in the United Kingdom [11], highlighting the potential user base for LearnCade.

Teachers are the secondary audience, as they can use the platform to engage entire classrooms in varied ways. Research shows that when learning is enjoyable, students are more motivated and better at retaining information [12]. Two detailed user personas were created to ensure LearnCade was designed around the real needs of its users.

User Persona 1 – Leo

Leo is a seven year old boy in Year 2 at primary school who represents LearnCade's primary end user. His love of games and curiosity to learn makes him the ideal user. A full breakdown of Leo's goals, behaviours and needs can be seen in Figure 3.1:



Figure 3.1 – Leo

User Persona 2 – Ms Grace

Ms Grace is a 30 year old primary school teacher with mixed ability classes. Her desire to use new technology and create memorable learning experiences makes her the ideal secondary user. A full breakdown of Ms Grace's goals, behaviours and needs can be seen in Figure 3.2:

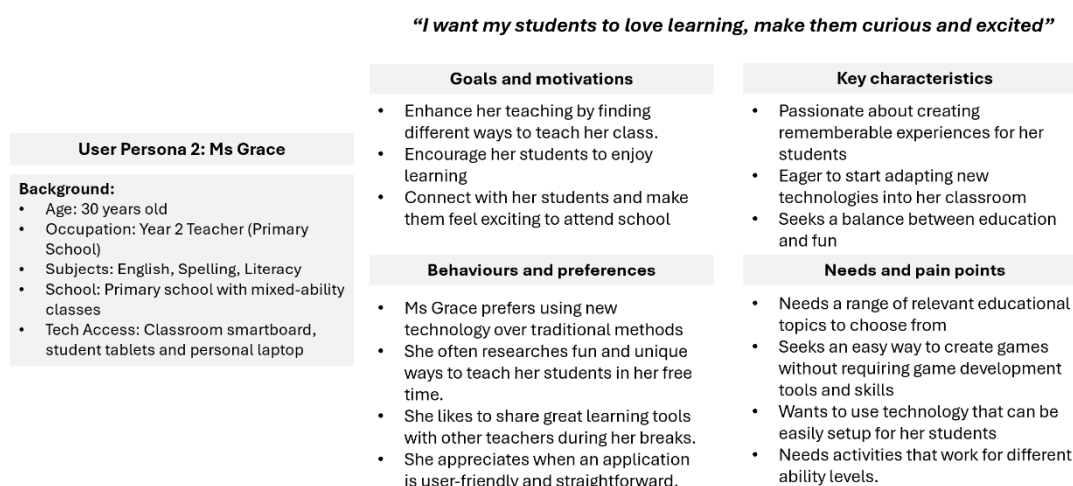


Figure 3.2 – Ms Grace

3.3 Customers

The primary customer group are parents of primary school children. Whilst children are the end users, parents are the decision makers when it comes to investing in the platform, and those who see genuine value are likely to upgrade to the 'premium plan'. Additionally, primary schools are another customer group as they may purchase an institutional licence, as discussed in Section 3.6. Initial outreach is planned through headteacher meetings as outlined in Section 3.5.4.

3.4 Lean Canvas

The Lean Canvas, shown in Figure 3.3, is a simple one-page planner that outlines the key elements of LearnCade's business idea:

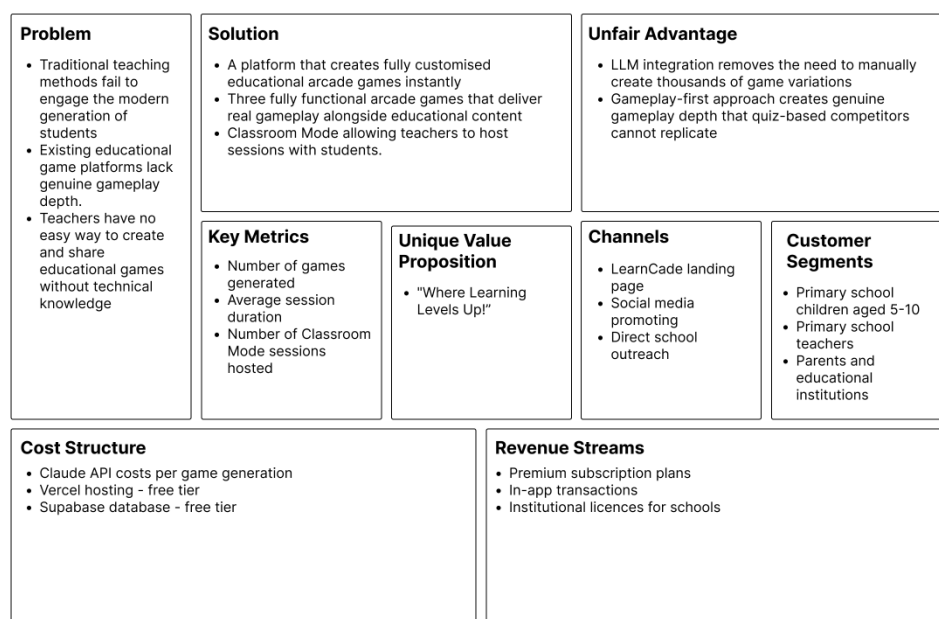


Figure 3.3 – LearnCade Lean Canvas, template taken from Figma Community [13]

3.5 The Market

The following section demonstrates understanding of LearnCade's market and competition:

3.5.1 Market Size and Research

LearnCade falls within the EdTech market, which involves digital products designed to support learning. According to a Department for Education report, the EdTech market in England generated £3.8 billion in gross value added in 2021 and was growing at 22% per year [14]. Schools also value products that include "options that could be tailored to individual learner needs" [14], which directly aligns with LearnCade's game creation flow.

Additionally, the success of a competitor further validates the market, according to Kahoot's official Q4 2022 financial report, the platform generated \$146 million in revenue with 60% year on year growth and over 8 million educators globally [15].

3.5.2 Ways to market

Figure 3.4 presents the key marketing methods for LearnCade. The main channels include social media gameplay snippets on YouTube, direct school outreach through headteacher meetings, word of mouth through the school community, launch events and email marketing to retain existing users.

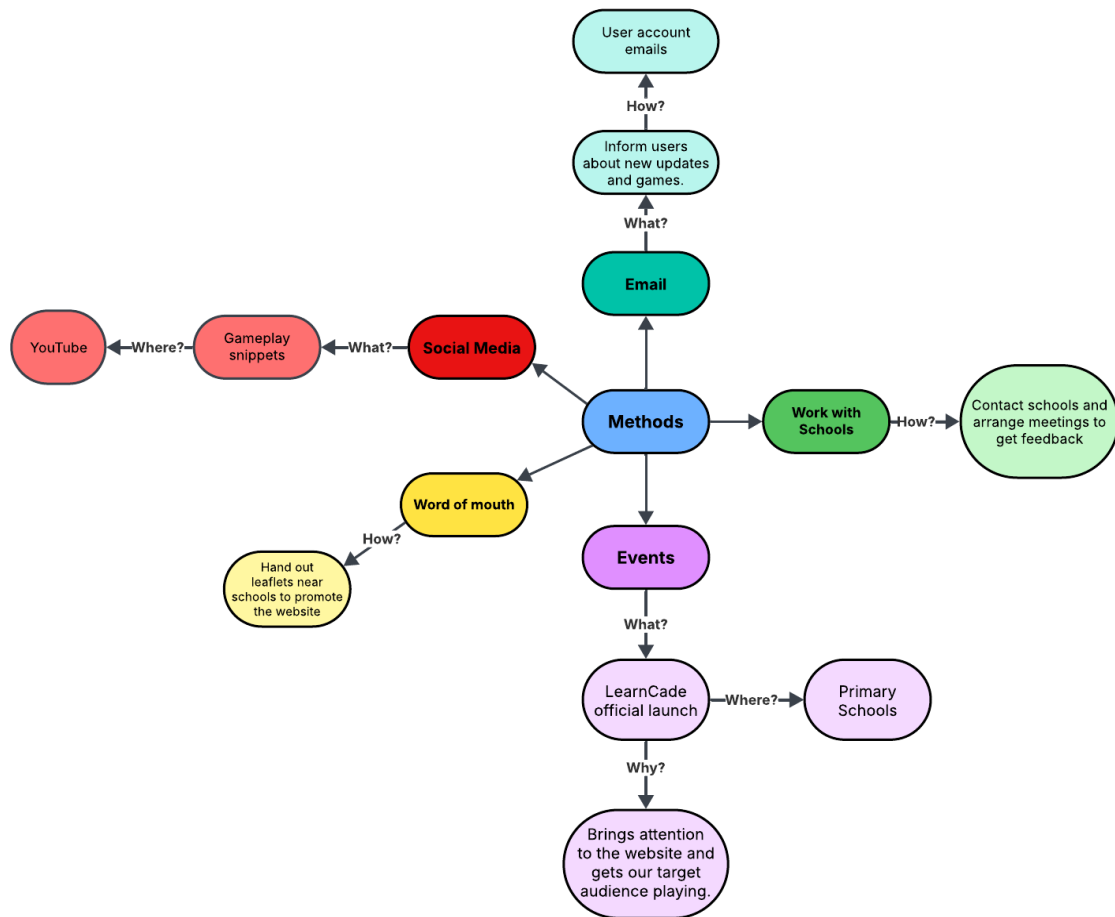


Figure 3.4 – Marketing Mind Map created using Lucidchart [16]

3.5.3 Competition

Direct Competitors:

- Educaplay - <https://www.educaplay.com/>
- Kahoot - <https://kahoot.com/>

Indirect Competitors:

- Funbrain - <https://www.funbrain.com/>
- BBC Bitesize - <https://www.bbc.co.uk/bitesize>
- Quizlet - <https://quizlet.com/latest>

The three most significant competitors are Kahoot, Educaplay and Funbrain. Kahoot and Educaplay are direct competitors as both use LLMs to deliver educational content. Funbrain is the most relevant indirect competitor, sharing the same target audience but without LLM generation or classroom multiplayer functionality. Figure 3.5 presents a strengths and weaknesses analysis of each:

Competitor	Strengths	Weaknesses
Kahoot	• Strong classroom multiplayer functionality (PIN-based) • A wide variety of topics that covers most age-ranges • High-quality student progress tracking (Tracks quiz scores and provides insights) • Clear revenue/business model • Partnerships with schools and universities worldwide	• Limited to quizzes, therefore, no real gameplay depth • When using the Kahoot Generator feature (LLM functionality), there is too much that is locked behind a pay wall.
Educaplay	• Intuitive creation flow • Strong customisation features • Wide variety of educational activities • Clear revenue/business model • Works with Google Classroom, Microsoft Teams, BlackBoard etc.	• Minimal gameplay depth with no engaging game mechanics • Forces users to create an account before creating a game. (Potentially causing low visit duration seen in Section 2.2.2) • The open text prompt functionality can lead to inconsistent, inappropriate and low quality generations.
Funbrain	• Includes a strong variety of different educational content (Games, Videos, and Books) • Allows user to filter by grade • Strong UI/UX, clearly designed for younger audiences and is easy to use • Includes a search functionality • All educational games are completely free • Strong revenue/business model as the platform attracts game developers to advertise their games on Funbrain to reach millions of students	• No usage of new technologies such as LLMs • Lacks multiplayer/classroom features • Gameplay varies, better depth than Educaplay and Kahoot but still lacks advanced game mechanics.

Figure 3.5 – Competition, Strengths and weaknesses table

After the analysis, LearnCade aims to turn their weaknesses into its core strengths. Where Kahoot and Educaplay are limited to quizzes with no real gameplay depth, LearnCade delivers three fully developed arcade games. Where Educaplay's open text prompt leads to inconsistent outputs, LearnCade uses a structured creation flow ensuring relevant, age-appropriate generations. Where Funbrain offers static content, LearnCade generates a unique personalised experience every time.

Figure 3.6 presents a feature comparison across the three most significant competitors and LearnCade, highlighting the gap in the market that LearnCade aims to fill.

Feature	Kahoot	Educaplay	Funbrain	LearnCade
Gameplay depth	No	No	Partial	Yes
Classroom Mode	Yes	No	No	Yes
Game sharing	Yes	Yes	No	Yes
Automated content generation	Yes	Yes	No	Yes
Structured creation flow	Partial	Yes	No	Yes
User profiles	Yes	Yes	No	Yes
Custom themes	Partial	Partial	No	Yes
UK education focus	No	No	No	Yes

Figure 3.6 – Feature comparison table

Figure 3.7 presents a Lean Canvas for Kahoot, showing their business model focuses on quiz sharing and PIN-based classroom sessions, with revenue from subscriptions, institutional licences and corporate training packages [17].

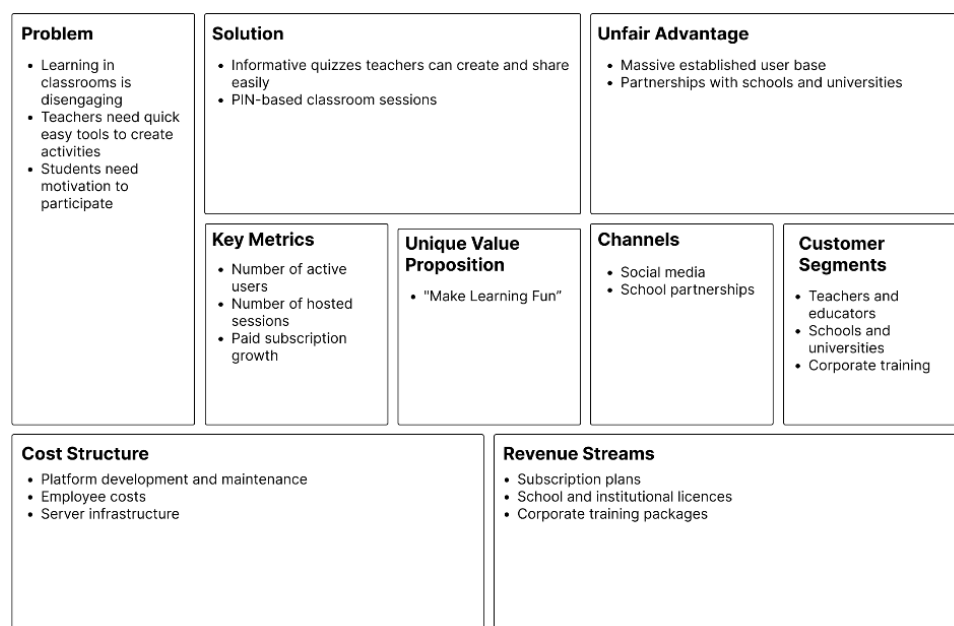


Figure 3.7 – Kahoot Lean Canvas, template taken from Figma Community [13]

3.5.4 Strategic Partnerships

As of now, no strategic partnerships have been established for LearnCade. However, given that the platform is designed for children aged 5-10 and teachers, primary schools represent the most valuable partnership opportunity.

The plan is to directly contact primary schools and arrange meetings with headteachers to demonstrate the platform and discuss the potential for LearnCade to be integrated into their school system.

A promising step towards this has already been taken, as the project supervisor is planning to arrange a meeting with a primary school headteacher to discuss LearnCade.

3.6 Revenue Model

The following section provides account of LearnCade's monetisation model as well as estimates of revenue and expenses:

3.6.1 Monetisation

1. Subscription Plans

LearnCade monetises through subscription plans, partially implemented in the platform. Users are assigned a free plan with limited daily generations, with the option to upgrade to a premium plan at £4.99 per month, unlocking unlimited generations and early access to new features.

Figure 3.8 shows the current pricing page.

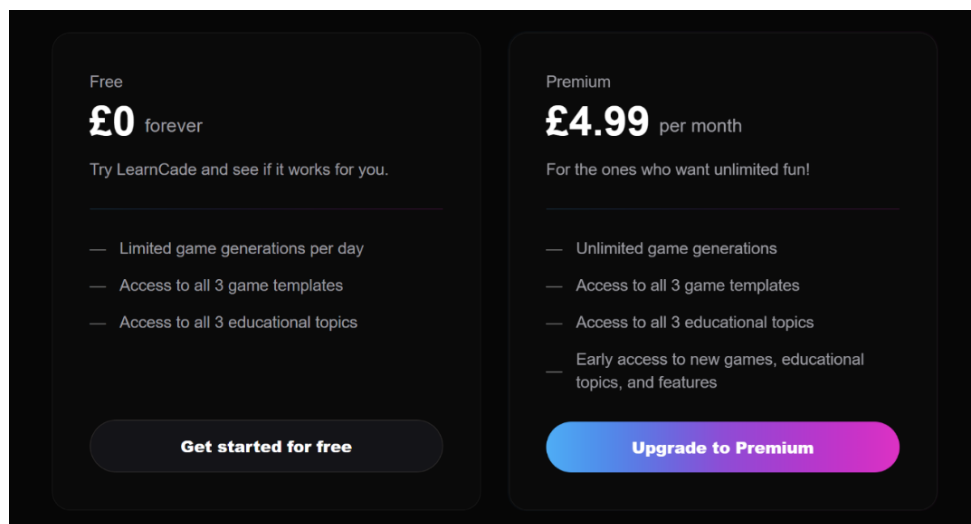


Figure 3.8 – LearnCade, Pricing

2. Microtransactions

Microtransactions will be available in the future of LearnCade. Additional game templates will be available at £4.99, offering new gameplay experiences. New templates could include a platformer inspired by Donkey Kong or a racing game inspired by OutRun. Character customisation items will also be available at £1.99, allowing users to purchase unique characters such as ninjas or knights.

3. Institutional Licences

LearnCade will sell annual institutional licences to primary schools at £999 per year. Based on an average of 268 pupils per school [11], individual premium subscriptions would cost schools around £16,044 per year, making the licence a significantly more cost-effective option. In comparison, Kahoot's EDU Standard plan costs £1,620 for just 10 host licences [18], whereas LearnCade's licence provides full access for all teachers and students.

3.6.2 Revenue and Expenses

Expenses

The following section outlines the estimated expenses for LearnCade based on a first year estimate of 10,000 users. All USD figures have been converted to GBP using the current exchange rate of \$1 = £0.76.

At 10,000 users, Vercel's free tier would no longer be sufficient and an upgrade to the Pro plan would be required at £15 per month or £180 per year [19].

The Claude API is the most significant ongoing cost, used to generate questions, answers and custom themes for every game. LearnCade uses the Claude Sonnet 4.6 model, priced at \$3 per million input tokens and \$15 per million output tokens [20]. Based on testing, a single game generation uses approximately 968 tokens at a cost of £0.0065. At an estimated 2,000 daily generations across 10,000 users, this amounts to £13 per day or £4,745 a year.

Supabase's free tier would also be insufficient at a scale of 10,000 users, requiring an upgrade to the Pro plan at £18.88 per month or £226 for the year [21].

The domain name 'learncade.online' is available through Hostinger [22] at £0.75 for the first year, however, prices increase for the following years.

Figure 3.9 summarises the estimated annual expenses:

Item	Annual Cost
Web Hosting (Vercel Pro)	£180
Claude API	£4,745
Database (Supabase Pro)	£226
Domain Name	£0.75
Overall	£5,152

Figure 3.9 – Annual expenses table

Revenue

The following section outlines the estimated revenue for LearnCade based on the same first year estimate of 10,000 users:

LearnCade's primary revenue stream is the premium subscription plan. According to Harvard Business Review, the average conversion rate for users to go from free to paid plans is 2-5% [23]. Therefore, an estimate of 3% of the user base (10,000 users) converting to premium would result in 300 subscribers. At £4.99, this would generate £1,497 per month or £17,964 per year.

Additionally, microtransactions offer another revenue stream for LearnCade. According to a study on microtransactions in the gaming industry, conversion rates for microtransaction purchases are between 5-6% [24]. Using this data, an estimated 5% of 10,000 users would purchase at least one game template at £4.99, generating £2,495. A further 5% are estimated to purchase at least one character customisation item at £1.99, generating £995. Together, this revenue stream would make £3,490

Institutional licences sold to schools represent the final revenue opportunity. Through school outreach and headteacher meetings, five institutional licences at £999 are estimated to be sold in the first year, making £4,995.

Figure 3.10 summarises the estimated annual revenue:

Item	Annual Cost
Subscription Plans	£17,964
Microtransactions	£3,490
Institutional Licences	£4,995
Overall	£26,449

Figure 3.10 – Annual revenue table

Estimated net profit (Year 1): Revenue (£26,449) – Expenses (£5,152) = **£21,297**

In summary, these figures demonstrate that LearnCade has a viable and profitable business model from its first year of operation, with all three revenue streams expected to grow significantly as the platform scales and acquires more school partnerships.

4. Design

This chapter outlines the key design decisions made during the development of LearnCade.

4.1 Design Software

The design tools used throughout the planning and visual design stages of LearnCade, are listed below:

- Figma [25] was selected as the wireframing tool for LearnCade. It allowed wireframes to be created, iterated and evaluated quickly as its intuitive interface and component system made it straightforward to design multiple pages and flows. Additionally, its widespread use in professional UI/UX design workflows meant that skills developed during this project can be directly transferred to industry.
- Krita [26] was chosen to create all the 2D assets for the arcade games, which included character sprites, enemies and powerups. As a free and open-source digital painting tool, Krita provided dedicated pixel art tools such as a pixel grid and frame-by-frame animation support, making it well suited for creating the retro-style assets that fit LearnCade's aesthetic.

4.2 User Flow

Before starting the visual design of the platform, a user flow was created to map out the key interactions a user would have with LearnCade from start to finish. This helped ensure the platform, particularly the game creation flow was designed in a clear and logical way.

Figure 4.1 presents the user flow from the perspective of Leo, the primary user persona introduced in Section 3.2.

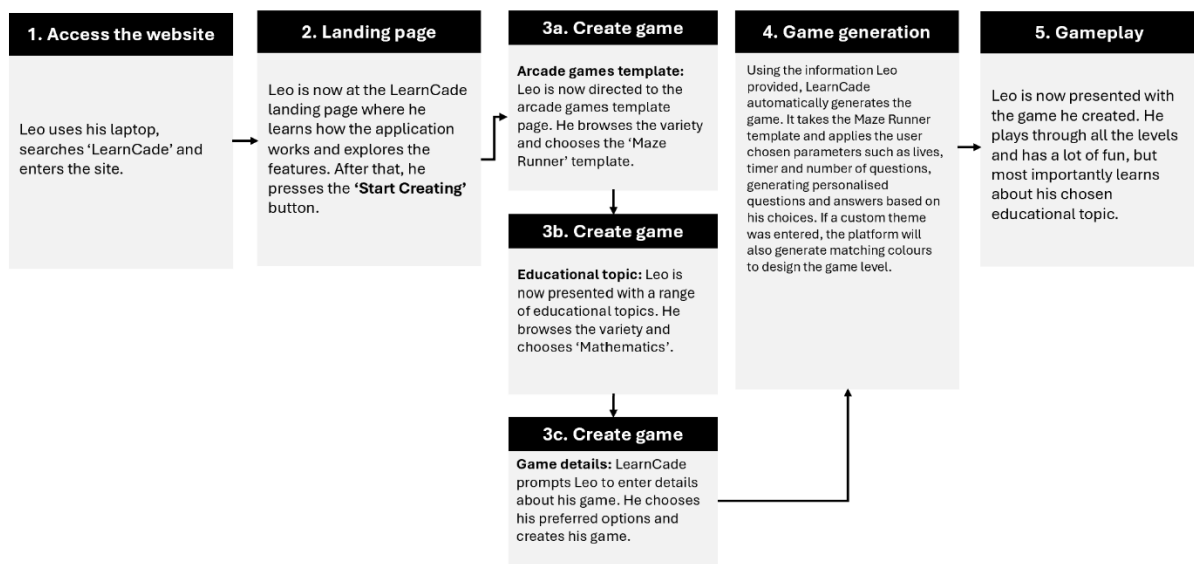


Figure 4.1 – User Flow Diagram

4.3 Logo Design

A strong logo is a fundamental part of any platform's identity, serving as the first visual representation. Figure 4.2 presents the LearnCade logo, designed in Figma.



Figure 4.2 – LearnCade Logo, designed for light and dark backgrounds

The logo is bold and easy to read, with a blue, purple, and pink gradient applied across the 'Cade' lettering to give it a vibrant and energetic feel that reflects the arcade nature of the platform. A row of pixel squares sit beneath the text, directly referencing the pixel art aesthetic of LearnCade. Additionally, I have created another version of the logo that can be set against a dark background, which allows the gradient colours to pop and ensures strong visual contrast.

4.4 Wireframes

Before any development began, wireframes were created using Figma to plan the layout and structure of LearnCade's main pages, ensuring the platform's structure was well thought out before implementation began.

Figure 4.3 presents the wireframes for the landing page, covering four sections, the hero section with the UVP tagline and video preview, a How it Works section, a Features card grid, and a Pricing section presenting the Free and Premium plans. A consistent navbar and footer are present throughout.

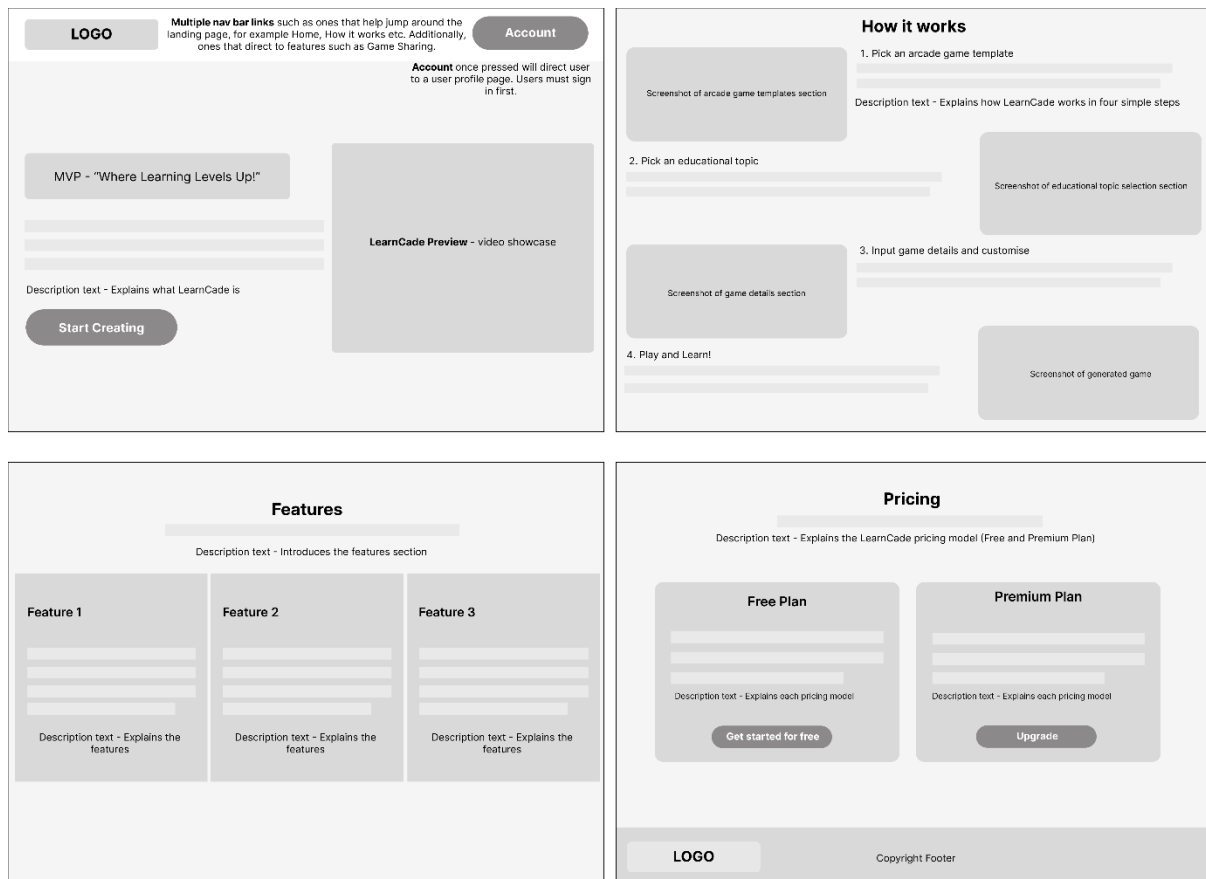


Figure 4.3 – Landing Page Wireframes

Figure 4.4 presents the three-step game creation flow. The first step displays the arcade game templates as cards, a layout influenced by Educaplay's template selection screen as discussed in Section 2.2.1. The second step presents the educational topic selection, and the third prompts users to enter game details through dropdowns and text inputs. This structured approach was a deliberate design decision to avoid overwhelming users with too many choices at once.

1. Choose your Arcade game template

Description text - Introduces the section

Small graphic of the game

Title - e.g Maze Runner

Description text - Explains each game template

Back Next

2. Select your educational topic

Description text - Introduces the section

Title - e.g Mathematics

Description text - Explains each educational topic

Back Next

3. Enter details about your game

Description text - Introduces the section

Option 1

Dropdown box - Users will be able to choose options to customise their game. For example, if the chosen topic was Mathematics, Option 1 may state 'Choose Operation'. The dropdown box will include Addition, Subtraction, Multiplication and Division.

Option 2

Custom option 1

Text box - Users will be able to type in here
Custom option 1 may state 'custom theme'

Create game

Figure 4.4 – Game Creation Flow Wireframes

Figure 4.5 presents the play page wireframe, showing the game canvas at the centre, a collapsible controls panel to the left, a game stats bar above the canvas and game configuration details in the top right.

User customisation settings: _____
For example: If they typed a custom theme it would be stated here _____

Question

Score: ____ Multiplier: ____ Lives: ____ Level: ____ Timer: ____

Controls

Description text - Explains the controls for the game.

This section may also include How to play, Game sharing, additional information about the powerups if necessary.

Arcade game output (HTML Canvas)

Figure 4.5 – Play Page Wireframe

Figure 4.6 presents the Game Share Code modal, featuring a text input for entering a six-character code and a Load Game button, accessible from the navbar across all pages.

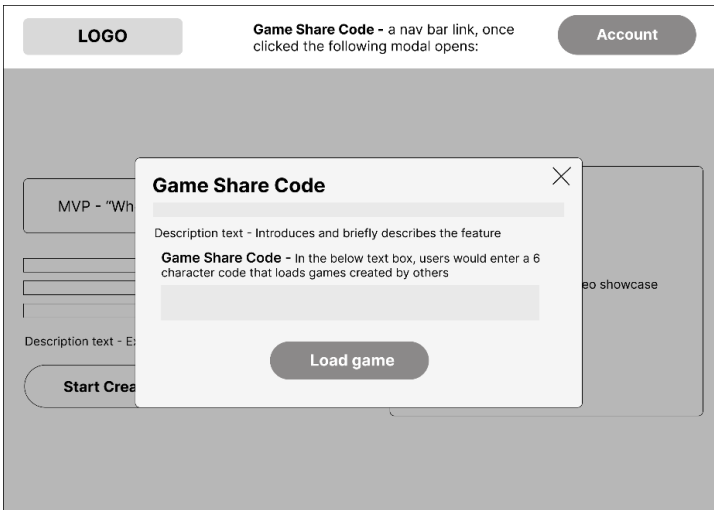


Figure 4.6 – Game Share Code Modal Wireframe

Figure 4.7 displays the Account Profile wireframes, showing summary stats, a Score Earned filter, Best Game and Recent Games sections, alongside a game overview modal with a Play Again button.

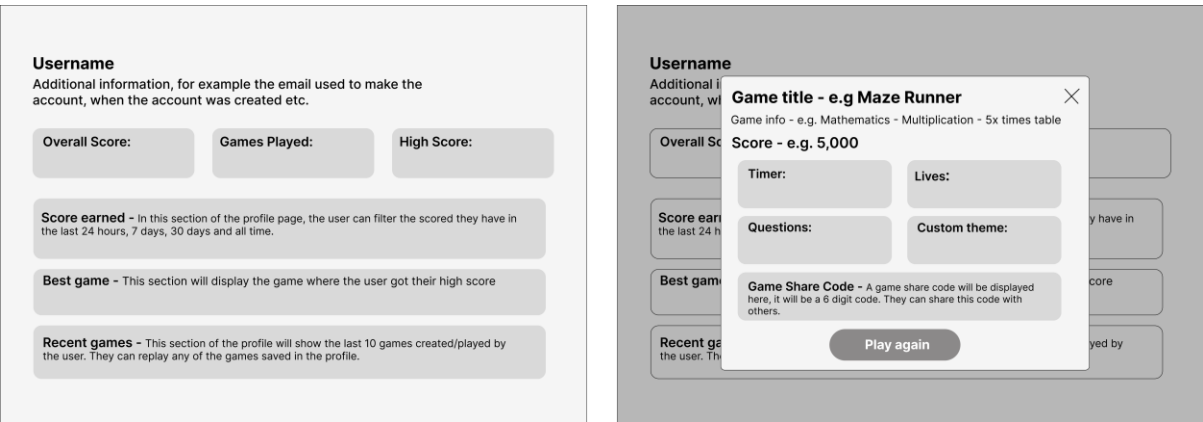


Figure 4.7 – Account Profile Wireframes

Lastly, Figure 4.8 shows the Classroom Mode wireframes, covering the full flow from the Host and Join modals through to the Waiting Room and live leaderboard.

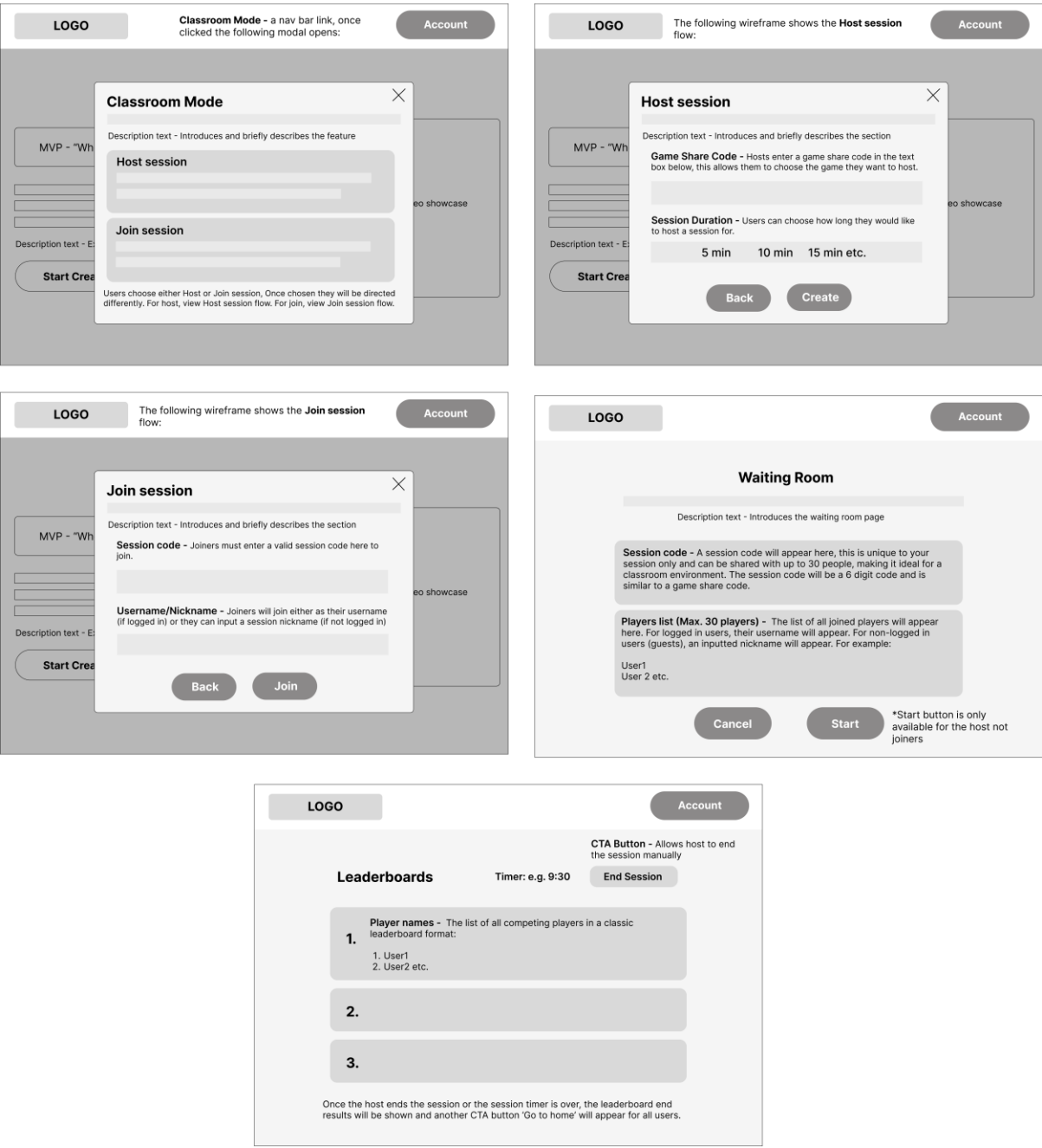


Figure 4.8 – Classroom Mode Wireframes

4.5 Game Design

The following section shows the design process behind the arcade games made for LearnCade.

1. Maze Runner

For the first game, Maze Runner, inspiration was drawn from the classic arcade title Pac-Man [27]. Its core loop of navigation, evasion and collection provides a strong framework for testing knowledge under pressure, and its familiar gameplay style ensures it gives users that strong arcade feeling.

To ensure the game felt unique from the original, I designed multiple assets using Krita. Figure 4.9 presents all the 2D pixel art assets created for Maze Runner.

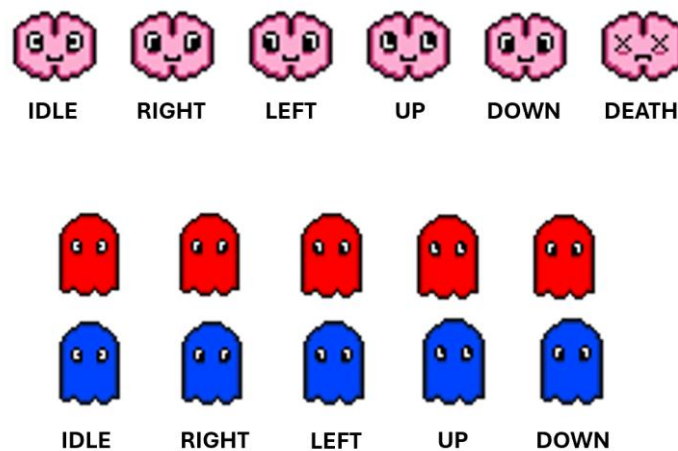


Figure 4.9 – Maze Runner Assets

The playable brain character was designed with six directional and state frames, idle, right, left, up, down and death, making the character feel alive.

The ghost NPCs were designed after the traditional enemies from Pac-Man, with five directional frames each and two colour variants, red and blue to make the game visually colourful and appealing. It is also worth noting that all assets are created at 33x32 pixels, ensuring consistency across all three games.

2. Space Shooter

The second game, Space Shooter, takes a more action focused approach. Space Invaders [28] was chosen as the foundation due to its fast-paced nature of shooting, dodging and eliminating incoming threats.

Figure 4.10 presents the pixel art assets created for Space Shooter.



Figure 4.10 –Space Shooter Assets

The playable character is a rocketship themed around the space setting, with the same brain character from Maze Runner visible inside the cockpit, reinforcing it as a recognisable mascot across the platform. Four frames were created, idle, right, left and death, where the death frame shows a fiery explosion to give the game visual impact.

The alien NPCs were designed after the classic Space Invaders enemies, each with a down and death frame, and produced in two colour variants, green and red to keep the game visually varied and appealing to younger players.

3. Brick Breaker

The third and final game offers a more strategic experience. Breakout [29] was chosen as the foundation, a game built around launching a ball and using a paddle to break through a wall of bricks.

Figure 4.11 presents the pixel art assets created for Brick Breaker.

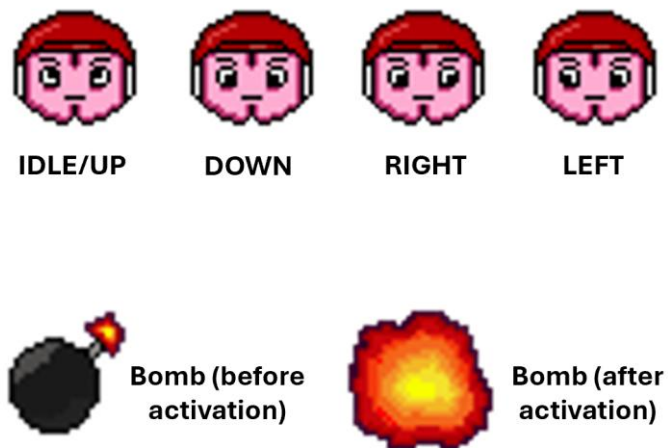


Figure 4.11 – Brick Breaker Assets

The brain character returns once more but with a distinctive red helmet, designed to represent the character protecting itself from the impact of the bricks and giving it a unique identity within this game. Four directional frames were created, idle/up, down, right and left.

A bomb power-up asset was also designed with two states, before and after activation, where the after state reuses the explosion frame seen in Space Shooter's death animation, connecting the asset library together.

4.6 Usability Considerations

The following section outlines the usability considerations for the LearnCade platform.

Colour and Contrast

Colour and contrast were carefully considered throughout the design of LearnCade. A dark background with bright, high-contrast text and UI elements was chosen to reflect the arcade aesthetic whilst also ensuring strong readability. The platform's brand gradient, seen in Section 4.3, was applied consistently across key interactive elements such as buttons.

Typography

Typography played an important role in enforcing LearnCade's identity. A pixel-style font, shown in Figure 4.12 was selected for headings and game-related elements to give the platform a strong arcade feel.



Figure 4.12 – LearnCade's Pixel Font (Press Start 2P by Google Fonts) [30]

Error Handling

Informative error messages were implemented across important user interactions, which can be viewed later in Section 5.8. The system would clearly display what went wrong and sometimes include buttons to direct away from issues, ensuring users particularly younger ones are not left confused when something goes wrong.

Navigation

A clear and structured navigation system was designed across LearnCade to ensure users can move between sections and features quickly and without confusion. The navbar is consistently present across all pages, providing links to key areas of the platform.

5. Implementation

This chapter details the implementation of LearnCade, covering the key technical decisions made during development and how the designs were brought to life.

5.1 System Architecture

Before exploring the actual implementation, it is important to understand the architecture of the platform and how its components interact. Figure 5.1 presents the LearnCade system architecture diagram.

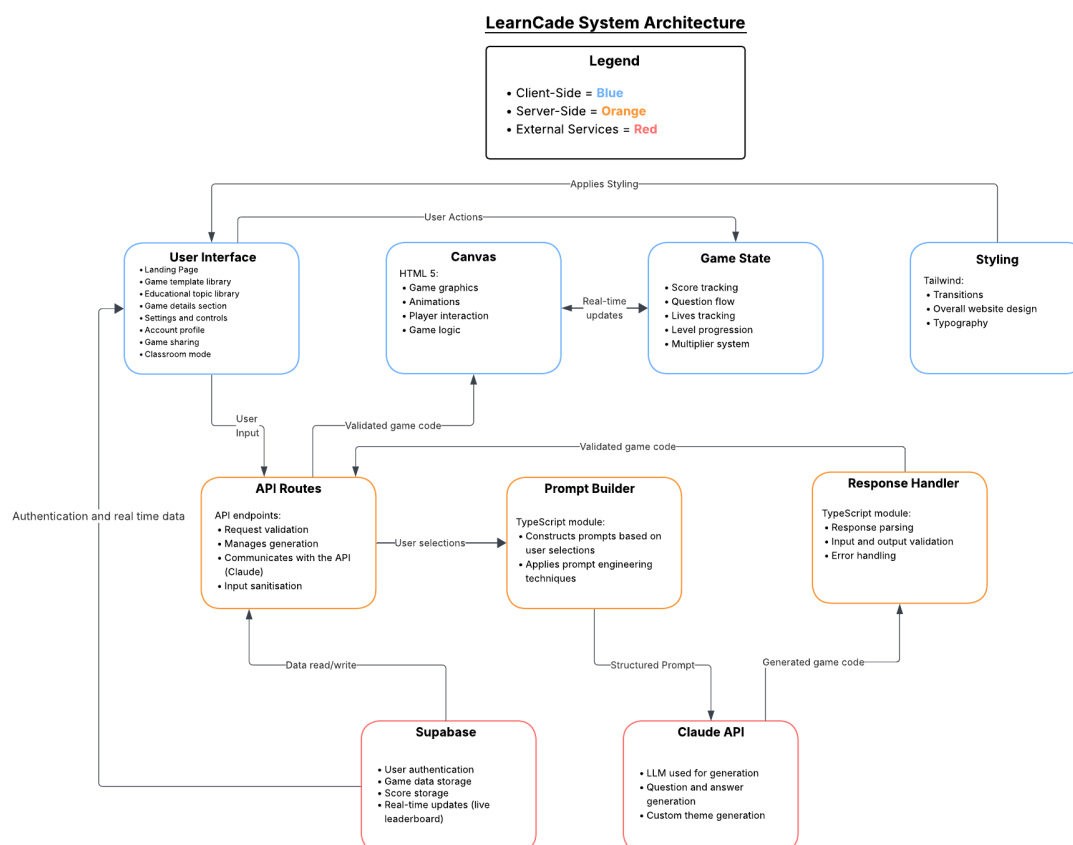


Figure 5.1 – LearnCade System Architecture (created using Lucidchart) [16]

The client-side layer consists of four key components. The User Interface handles all the front-end pages and interactions across the platform, from the landing page and game creation flow through to the account profile and classroom mode. The HTML5 Canvas is responsible for rendering the arcade games, handling graphics, animations, player interaction and game logic. The Game State manages all real-time data during gameplay including score tracking, question flow, lives, level progression and the multiplier system, communicating with the Canvas through continuous updates. Finally, Tailwind CSS is applied globally across all components to ensure a consistent visual style, handling transitions, typography and the overall website design.

The server-side layer is built entirely within Next.js and is made up of three modules. The API Routes serve as the entry point for all server-side requests, handling request validation, managing the generation process, communicating with the Claude API, and applying input sanitisation. The Prompt Builder is a TypeScript module that takes the user's game selections and constructs prompts before passing them to the Claude API. The Response Handler then receives the generated output, parsing the response, validating inputs and outputs, and handling any errors before the validated game code is returned to the client.

At the external services layer, the Claude API receives the prompt and generates the educational questions, answers and custom theme content that powers each game session. Supabase handles user authentication, game data storage, score storage and real-time data that drive the live leaderboard in Classroom Mode.

5.2 API Integration

The Claude API is the core technology that powers LearnCade's educational content generation, dynamically generating questions, answers and custom themes for each game based on the user's selections.

When a user completes the game creation flow and clicks Create Game, a POST request is sent to the API route. The route takes the user's selections from the request body as shown in Figure 5.2, capturing the game template, educational topic, operation or category, number of questions and optional custom theme, which are then used to construct a prompt.

```
export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const {
      gameTemplate,
      topic,
      operation,
      timesTable,
      numberRange,
      spellingCategory,
      scienceCategory,
      numberOfQuestions,
      customTheme,
    } = body;
  }
```

Figure 5.2 – POST request code snippet

Prompt Builder:

The prompt logic in LearnCade, referred to in the System Architecture as the 'Prompt Builder', is the most significant part of the integration. The route builds a structured prompt based on the user's exact selections, with different logic for each of the three topics.

Throughout all prompts, the phrase "for children aged 5-10" is consistently included to ensure Claude's understanding of the target audience and appropriate difficulty level, a technique known as context prompting, where information is provided to guide the model's behaviour without explicit examples.

For Mathematics, the prompt is built based on the operation selected in the creation flow. Multiplication and Division share the same underlying structure, both use a times table selection, with Division presented as 'Divide by' options in the UI to make it more intuitive for users. If the user selects 'mixed', Claude is instructed to draw from any times table between 1 and 12 randomly. Figure 5.3 shows the prompt constructed for Multiplication with Mixed selected.

```
if (topic === "mathematics") {
  if (operation === "multiplication" && timesTable) {
    if (timesTable === "mixed") {
      questionsPrompt = `Generate ${numberOfQuestions} multiplication questions for children aged 5-10.
      Use any times tables from 1 to 12.
      Mix different number combinations randomly (e.g., 7 x 8, 3 x 6, 11 x 4, 5 x 9).
      Format: "X x Y = ?";`;
```

Figure 5.3 – Mathematics Prompt Builder code snippet

Addition and Subtraction follow a different pattern, enforcing a number range rather than a times table. The range is parsed from the user's input and placed into the prompt to set both the minimum and maximum values for the numbers used, with subtraction constrained to ensure the answers are positive, although, support for negative numbers is identified as a potential area for future development. Figure 5.4 shows the prompt constructed for Addition:

```
} else if (operation === "addition" && numberRange) {
  const [min, max] = numberRange.split("-").map(Number);
  questionsPrompt = `Generate ${numberOfQuestions} addition questions for children aged 5-10.
  Both numbers must be between ${min} and ${max}. The answer must not exceed ${max}.
  Mix different number combinations randomly.
  Format: "X + Y = ?";`;
```

Figure 5.4 – Multiplication, Mixed Prompt Builder code snippet

For Spelling, a category guide object approach was used, providing Claude with topic based constraints. Figure 5.5 shows the Spelling category guide for Spelling, Animals:

```
const spellingGuides: Record<string, { hint: string; good: string; bad: string }> = {
  animals: {
    hint: "Use recognizable animals children learn about: big cats (tiger, leopard), sea creatures (dolphin, octopus, whale),
    good: "tiger, elephant, giraffe, dolphin, penguin, crocodile, butterfly, kangaroo, leopard, cheetah, gorilla, rhinoceros,
    bad: "ant, bee, fly, rat, bat",
  },
};
```

Figure 5.5 – Spelling, Animals Prompt Builder code snippet

This prompt applies two distinct techniques. The good examples list is a demonstration of few-shot prompting, providing Claude with examples of the expected output to guide its output. The bad list applies negative prompting, explicitly instructing Claude to avoid certain outputs, in this case overly simple words, steering the model away from unwanted results.

A similar approach was used for Science, where the 'scienceGuides' object includes descriptions and example questions to constrain Claude strictly to the chosen category. Figure 5.6 shows the Science, Human body category prompt, demonstrating zero-shot prompting, as the task and constraints are described in detail without providing worked examples of every possible question type:

```
"human-body": {  
  focus: "Focus on body systems, organs, and functions. Examples: heart, lungs, brain, stomach, liver,  
  examples: `Good questions: \"What organ pumps blood?\" (heart), \"What do you use to breathe?\" (lungs),  
  Avoid: questions not specifically about body parts or systems.`",
```

Figure 5.6 – Science, Human Body Prompt Builder code snippet

Output formatting:

All three topics instruct Claude to return responses as a strict JSON array with no additional text or markdown, an approach known as constrained output prompting. Figure 5.7 shows the output formatting used for the Science topic.

```
Return ONLY a JSON array with this exact format, no other text or markdown:  
[  
  {  
    "question": "Which planet is closest to the sun?",  
    "correct": "Mercury",  
    "wrong": ["Venus", "Mars", "Earth"]  
  }  
]
```

Figure 5.7 - Science, output formatting code snippet

The JSON structure also requires Claude to generate both the correct answer and three wrong answers for every question, with wrong answers required to be relevant alternatives from the same category, as seen in the example where "Mercury" is the correct answer and "Venus", "Mars" and "Earth" are the wrong options. This is another instance of few-shot prompting, where the example of the expected structure guides Claude to produce consistently valid outputs.

Each prompt also includes a set of rules to ensure Claude's output is consistent, accurate and appropriate for the platform. Figure 5.8 shows the rules defined for the Spelling topic:

```
Rules:  
- Use lowercase for all words except proper nouns (countries)  
- Countries must be capitalised (France, England, Spain)  
- Use British English spelling (colour not color, favourite not favorite)  
- All words must be interesting and relevant for children aged 5-10  
- No plurals, no compound words with hyphens  
- Mix up difficulty randomly  
- No duplicate words`;
```

Figure 5.8 – Spelling, output rules code snippet

These rules are a critical part of the output, as they enforce requirements that Claude would not know about. For example, enforcing British English spelling ensures the platform is appropriate for its intended UK audience, without this rule, Claude defaults to American English spelling such as "color" and "favorite" which would be marked as incorrect in a UK classroom.

Furthermore, capitalisation rules for proper nouns such as countries ensure grammatical correctness in the generated output. Rules preventing plurals, compound words and duplicate words further constrain the output to ensure the generated words are clean and suitable for both the target audience and the game's user interface.

For both Spelling and Science, which are presented to the user in a multiple choice format (MCQ), the validated answers are arranged into the structure used in the game. As shown in Figure 5.9, the correct answer and three wrong answers are combined into a single array and shuffled randomly, before being mapped to letter labels A, B, C and D. The index of the correct answer is tracked before shuffling so the system always knows which letter is correct regardless of where it lands after the shuffle.

```
} else if (topic === "science") {
  questions = rawResponse.map((item: { question: string; correct: string; wrong: string[] }) => {
    const correctOption = item.correct;
    const allChoices = [correctOption, ...item.wrong.slice(0, 3)].sort(() => Math.random() - 0.5);
    const correctIndex = allChoices.indexOf(correctOption);
    const letters = ["A", "B", "C", "D"];

    return {
      question: item.question,
      choices: allChoices,
      correct: letters[correctIndex],
      wrong: letters.filter((_, i) => i !== correctIndex),
    };
  });
}
```

Figure 5.9 – Science, MCQ output formatting

Response Handling and Validation:

Once Claude returns a response, the output is processed and validated before being sent, this process is showcased as the 'Response Handler' in the System Architecture diagram. LLM output can be unpredictable, making response handling and validation an important part of the integration.

Despite being explicitly instructed to return only JSON, Claude occasionally wraps its response in markdown formatting. To handle the possible formatting variations, a stripping step was implemented as shown in Figure 5.10, which detects and removes the markdown formatting before the response is parsed. Without this step, the application would throw an error, and the game generation would fail entirely.

```
let responseText = questionsContent.text.trim();
if (responseText.startsWith("```json")) {
  responseText = responseText.replace(/^```json\s*/, "").replace(/```\s*$/, "");
} else if (responseText.startsWith("```")) {
  responseText = responseText.replace(/^```\s*/, "").replace(/```\s*$/, "");
}
```

Figure 5.10 – Stripping Step code snippet

Even after parsing, Claude does not always return perfectly structured data. To handle this, each topic applies its own validation logic to catch and fix any issues before the questions are sent to the client. Figure 5.11 shows the validation applied for Mathematics:

```
let offset = 1;
while (uniqueWrong.length < 3 && offset < 20) {
  const newWrong = correct + offset;
  if (!uniqueWrong.includes(newWrong) && newWrong > 0) {
    uniqueWrong.push(newWrong);
  } else {
    const altWrong = correct - offset;
    if (!uniqueWrong.includes(altWrong) && altWrong > 0) {
      uniqueWrong.push(altWrong);
    }
  }
  offset++;
}

return {
  question: q.question || `${correct} = ?`,
  correct,
  wrong: uniqueWrong,
};
});
```

Figure 5.11 – Mathematics, response handling and validation code snippet

As seen in the example, the handler first checks that all wrong answers are unique and different from the correct answer. A fallback was also added for cases where Claude returns fewer than three valid wrong answers. The system attempts to generate the remaining values by adding to the correct answer, and if that value is already taken or invalid, it tries subtracting instead, continuing until three valid wrong answers are found. This ensures the game always has exactly four answer options regardless of what Claude returns.

For Spelling, rather than relying on Claude to generate misspellings, wrong answer options are generated using a ‘generateWrongSpellings’ function. As shown in Figure 5.12, the function works by splitting the word into individual letters and applying vowel swapping, replacing each vowel with a close alternative to produce a realistic but incorrect spelling.

```
function generateWrongSpellings(word: string): string[] {
  const wrong: string[] = [];
  const letters = word.split("");

  const vowelSwaps: Record<string, string> = {
    a: "e", e: "a", i: "e", o: "a", u: "o",
  };

  for (let i = 0; i < letters.length && wrong.length < 3; i++) {
    const swap = vowelSwaps[letters[i].toLowerCase()];
    if (swap) {
      const modified = [...letters];
      modified[i] = swap;
      const wrongWord = modified.join("");
      if (wrongWord !== word && !wrong.includes(wrongWord)) {
        wrong.push(wrongWord);
      }
    }
  }
}
```

Figure 5.12 – Spelling, response handling and validation code snippet

Custom theme generation:

If the user enters a custom theme during the game creation flow, a second API call is made after the questions have been generated. The theme prompt instructs Claude to generate a colour scheme that matches the user's input, returning a JSON object containing hex colour values for the maze walls and pellets. Figure 5.13 shows the theme prompt construction:

```
if (customTheme) {
  const themePrompt = `The user wants a "${customTheme}" theme for their ${gameTemplate} game.
  Generate a colour scheme that matches the user inputted theme.

  Return ONLY a JSON object with this exact format, no other text or markdown:
  {
    "wall": "#hexcolor",
    "pellet": "#hexcolor",
    "background": "□ #000000"
  }

  Rules:
  - Background MUST always be "□ #000000" (black)
  - Wall colour should be bright and vibrant to stand out against black
  - Wall and pellet must have good contrast between each other
  - If user mentions one colour: use different shades/tints of that colour for wall and pellet
  - If user mentions two colours: use first for walls, second for pellets
  - Use vivid, saturated colours that pop on black backgrounds
  - Examples: red = □ #FF0000, blue = □ #0000FF
  - Use □ #01ae01 for green, not □ #00FF00 (too bright)
  - Use the pure hex version of a colour unless the user specifies a shade (e.g. dark red, light blue)
  - Never use dark colours for walls or pellets as they will not show on a black background
  - Always use hex format with # symbol`;
```

Figure 5.13 – Custom theme generation prompt and output formatting

The prompt includes rules to ensure the generated colours work visually within the game, the background is always enforced as black to maintain the arcade aesthetic, and wall and pellet colours must be bright and vibrant enough to stand out against it. These rules ensure Claude returns colours that are immediately usable.

5.3 Games

The following chapter details the implementation of each of the three arcade games in LearnCade.

1. Maze Runner

Maze Runner is the first of the three games, inspired by Pac-Man. Before the game begins, an objective and controls modal appears as shown in Figure 5.14, helping players fully understand the objective and controls before pressing start, after which the background music plays and the game begins.

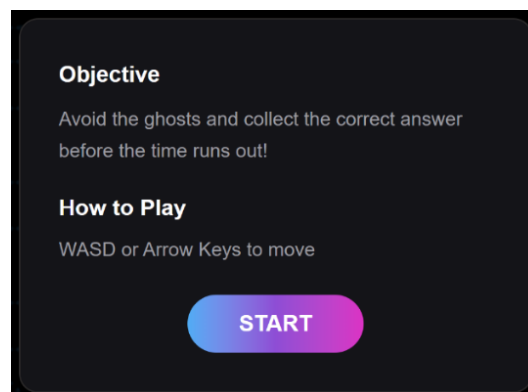


Figure 5.14 – Maze Runner, start modal

Figure 5.15 shows the finished Maze Runner game, in this example the user has created a Mathematics game focused on the five times table, with a 40 second timer, 10 levels and 3 lives. The questions and answers generated by the Claude API are passed directly into the game, with the correct answer and three wrong answers placed into the four answer circles.



Figure 5.15 – Maze Runner

Gameplay

The player controls the brain character using WASD or the arrow keys, navigating the maze to find and collect the correct answer. Four answer circles are placed across the maze at the start of each level, one in each quadrant, and the player must collect the correct answer to complete the level. Two +100 power-up collectibles are also placed in the maze, collecting them is a risk-reward decision as it costs time and increases the chance of being spotted by a ghost.

Each correct answer awards a base score determined by the difficulty setting, with harder time limits awarding more points. This base score is multiplied by the current multiplier, which starts at 1.0x and increases by 0.1x with every correct answer, resetting to 1.0x if the player answers incorrectly or is caught. A completion bonus of 500 points is awarded for finishing all levels.

There are three ways to lose a life, collecting the wrong answer, being caught by a ghost, or running out of time. If all lives are lost the game ends.

Level Design

Maze Runner's level is created directly in the code as an array of 0s and 1s, as shown in Figure 5.16. Every 1 represents a wall and every 0 represents a walkable path. If a cell contains a 1, nothing can move into it. This approach meant the entire level could be designed and adjusted in code without any external tools or assets.

```
const MAZE = [
  [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
  [1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,1],
  [1,0,1,1,0,1,1,1,0,1,0,1,1,0,1,1],
  [1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1],
  [1,0,1,1,0,1,0,1,1,1,1,0,1,0,1,1],
  [1,0,0,0,0,1,0,0,0,1,0,0,0,1,0,0,1],
  [1,1,1,1,0,1,1,1,0,1,0,1,1,0,1,1,1],
  [1,1,1,1,0,1,0,0,0,0,0,0,0,1,0,1,1,1],
  [0,0,0,0,0,0,0,1,1,0,1,1,0,0,0,0,0,0],
  [1,1,1,1,0,1,0,0,0,0,0,0,0,1,0,1,1,1],
  [1,1,1,1,0,1,0,1,1,1,1,1,0,1,0,1,1,1],
  [1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1],
  [1,0,1,1,0,1,1,1,0,1,0,1,1,0,1,1,0,1],
  [1,0,0,1,0,0,0,0,0,0,0,0,0,0,0,1,0,0,1],
  [1,1,0,1,0,1,0,1,1,1,1,0,1,0,1,0,1,1],
  [1,0,0,0,0,1,0,0,0,1,0,0,0,1,0,0,0,0,1],
  [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
];
```

Figure 5.16 – Maze Runner, level design code snippet

Ghost Logic

The ghost logic was one of the most technically interesting aspects of Maze Runner to implement. Each ghost operates in one of two modes, patrol and chase. In patrol mode, ghosts move around the maze while gradually drifting toward the answer circles to keep pressure on the player. When a ghost comes within a detection radius of three cells from the player, it switches to chase mode, increases its speed and actively moves toward the player. Figure 5.17 shows the chase detection logic, where the distance between the ghost and player is calculated. Chase mode lasts five seconds before the ghost returns to patrol, giving the player a chance to escape.

```
const player = playerRef.current;
const distToPlayer = getDistanceToTarget(ghost.x, ghost.y, player.x, player.y);

if (distToPlayer <= CHASE_DETECTION_RADIUS) {
  if (ghost.mode !== "chase") {
    ghost.mode = "chase";
    ghost.chaseStartTime = Date.now();
    ghost.targetPellet = null;
    playSound(sfxGhostDetectRef);

    const dx = player.x - ghost.x;
    const dy = player.y - ghost.y;
    const dirs = ["up", "down", "left", "right"];
    const opposite: Record<string, string> = { up: "down", down: "up", left: "right", right: "left" };
    const validDirs = dirs.filter(dir => {
      let tx = ghost.x;
      let ty = ghost.y;
      if (dir === "up") ty -= 1;
      else if (dir === "down") ty += 1;
      else if (dir === "left") tx -= 1;
      else if (dir === "right") tx += 1;
      return canMove(tx, ty) && dir !== opposite[ghost.direction];
    });
  }
}
```

Figure 5.17 – Maze Runner, Ghost logic code snippet

Visual Feedback

LearnCade uses visual and audio feedback throughout to make every interaction feel responsive. For example, when a ghost enters chase mode, a red glow appears and a sound effect plays as shown in Figure 5.18.

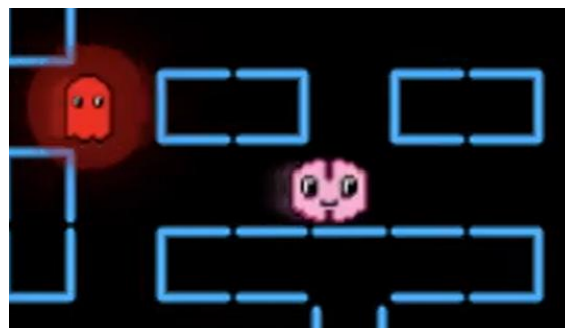


Figure 5.18 – Maze Runner, ghost visual feedback

Collecting a power-up displays floating +100 score text as shown in Figure 5.19. Additionally, collecting the correct answer triggers green particles and floating score text, whilst the wrong answer triggers the same in red.

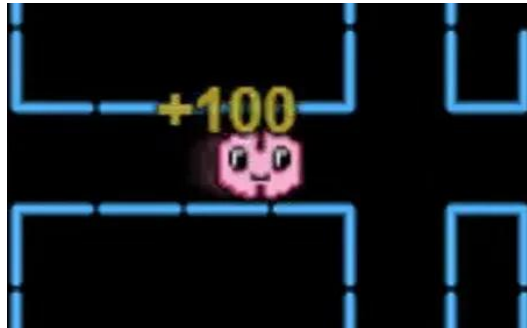


Figure 5.19 – Maze Runner, powerup visual feedback

When a life is lost, the brain character switches to its death frame and the arena flashes red as shown in Figure 5.20.

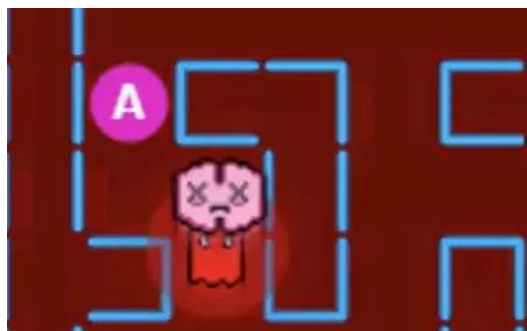


Figure 5.20 – Maze Runner, death visual feedback

2. Space Shooter

Space Shooter is the second game, inspired by Space Invaders. Before the game begins, an objective and controls modal appears as shown in Figure 5.21, briefing the player on the objective and controls before they press start, after which the background music plays and the game begins.

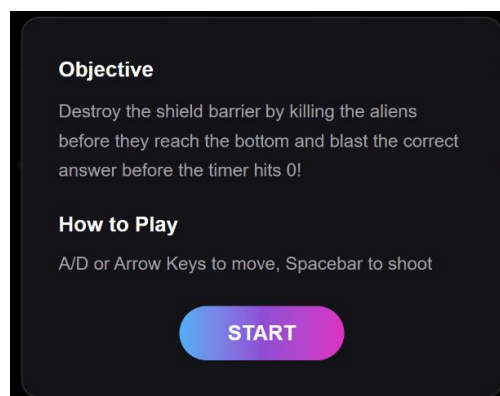


Figure 5.21 – Space Shooter, start modal

Figure 5.22 shows the finished Space Shooter game, in this example the user has created a fun Spelling game about countries with a custom theme of green and blue. Rather than numerical answer circles, the game displays a multiple choice question with four labelled options A, B, C and D, with the corresponding answer circles displayed inside the arena. This MCQ format is only used for Spelling and Science topics.

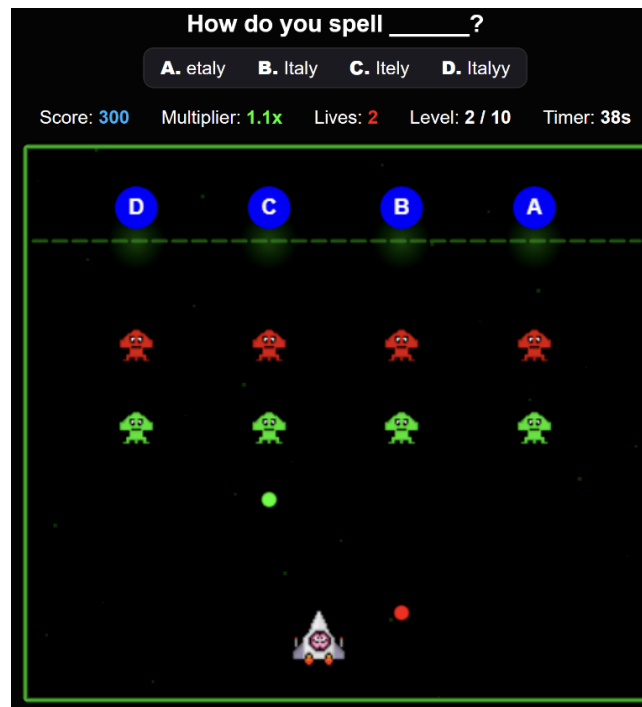


Figure 5.22 – Space Shooter

Gameplay

The player controls the rocketship using the left and right arrow keys or A/D, and the spacebar to shoot. Two waves of aliens descend toward the player at the start of each level, four green in the front row and four red behind. Red aliens award 15 points when shot and green aliens award 10. As the waves descend, aliens fire bullets downward requiring the player to dodge whilst clearing the waves. A laser shield prevents the answer circles from being reached until all aliens are cleared. Once cleared, a +100 power-up spawns before the answer circles become accessible.

Shooting the wrong answer circle, being hit by an alien bullet, allowing an alien to reach the bottom, or running out of time all result in losing a life. The same scoring and multiplier system from Maze Runner applies throughout.

Level Design

Unlike Maze Runner which uses a 2D array to define the playable space, Space Shooter's arena is constructed directly in the Canvas draw function. As shown in Figure 5.23, a glowing border is drawn around the edges of the canvas to define the play area boundary, and a star field is created by scattering 30 stars of varying sizes across the background using fixed position offsets, creating the space aesthetic without any external assets or image files.

```
const draw = (ctx: CanvasRenderingContext2D) => {
  ctx.fillStyle = COLORS.background;
  ctx.fillRect(0, 0, CANVAS_WIDTH, CANVAS_HEIGHT);

  for (let i = 0; i < 30; i++) {
    const x = (i * 37 + 13) % CANVAS_WIDTH;
    const y = (i * 59 + 29) % CANVAS_HEIGHT;
    const size = 1 + (i % 3);
    ctx.fillStyle = COLORS.accent + "40";
    ctx.fillRect(x, y, size, size);
  }

  ctx.strokeStyle = COLORS.accent;
  ctx.lineWidth = 3;
  ctx.lineCap = "round";
  ctx.strokeRect(4, 4, CANVAS_WIDTH - 8, CANVAS_HEIGHT - 8);
}
```

Figure 5.23 – Space Shooter, level design code snippet

Alien Shooting

The alien shooting system was one of the most interesting aspects of Space Shooter to implement. Rather than firing at fixed intervals, aliens shoot periodically based on a frame counter, triggering a shot attempt every 40 frames. At each trigger point, a random alive alien is selected from the wave and fires a bullet downward toward the player, with the bullet colour matching the colour of the alien that fired it, red or green, adding visual variety to the incoming fire. A probability check of 55% is also applied at each trigger, ensuring shots feel unpredictable, as shown in Figure 5.24.

```
const alienShoot = () => {
  if (isPausedRef.current) return;
  if (gameStateRef.current !== "playing") return;
  if (showLevelCompleteRef.current || showLifeLostRef.current) return;

  alienShootTimer.current++;
  if (alienShootTimer.current > 40) {
    alienShootTimer.current = 0;
    const aliveAliens = aliensRef.current.filter(a => a.alive);
    if (aliveAliens.length > 0 && Math.random() < 0.55) {
      const shooter = aliveAliens[Math.floor(Math.random() * aliveAliens.length)];
      alienBulletsRef.current.push({
        x: shooter.x,
        y: shooter.y,
        color: shooter.color === "green" ? "#00ff00" : "#ff0000",
      });
    }
  }
};
```

Figure 5.24 – Space Shooter, laser shield code snippet

Visual Feedback

Space Shooter carries over the same visual and audio feedback system with a few additions. Each shot fires a red bullet with a gunshot sound effect as shown in Figure 5.25.



Figure 5.25 – Space Shooter, bullet visual feedback

Each alien killed displays floating score text as shown in Figure 5.26



Figure 5.26 – Green and Red aliens, visual feedback

When the player dies the rocketship switches to its explosion frame as shown in Figure 5.27

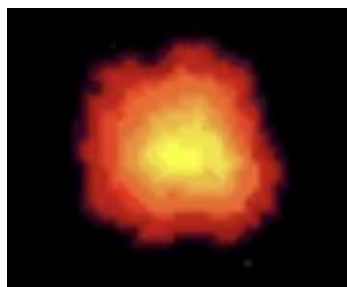


Figure 5.27 – Explosion visual feedback

Power-up collection and correct or incorrect answer circles trigger the same effects as Maze Runner.

3. Brick Breaker

Brick Breaker is the third and final game, inspired by Breakout. In similarity, an objective and controls modal appears before the game begins, as shown in Figure 5.28. This briefs the player on the objective and controls before they play, after which the background music starts and the game begins.

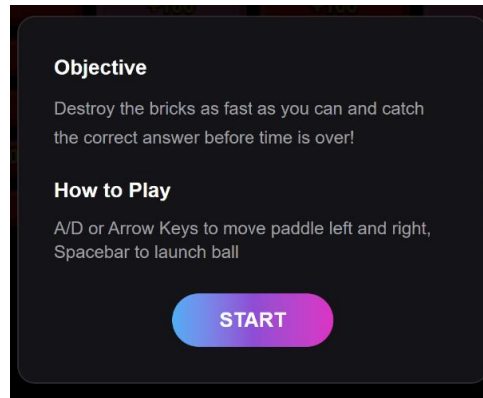


Figure 5.28 – Brick Breaker, start modal

Figure 5.29 shows the finished Brick Breaker game, in this example the user has created a Science game about the human body with a red and pink custom theme.

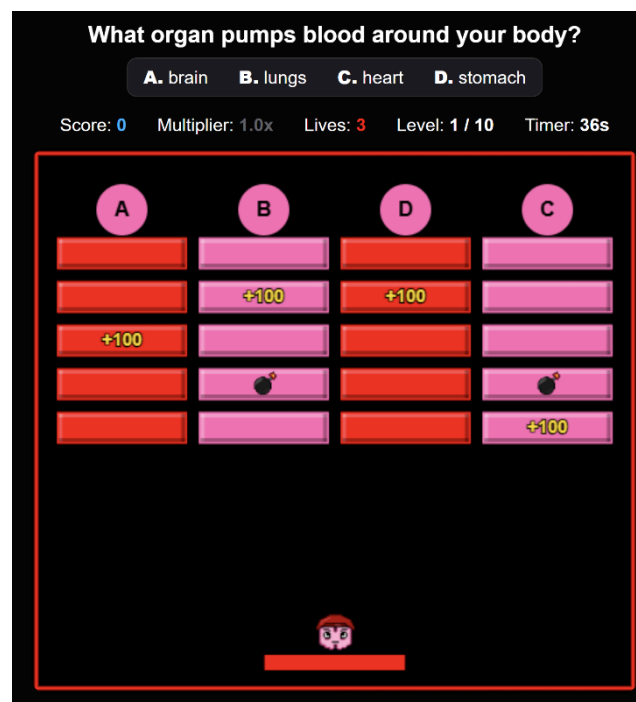


Figure 5.29 – Brick Breaker

Gameplay

The player controls the paddle using the left and right arrow keys or A/D, and presses spacebar to launch the ball. There are four columns of five bricks, each with an answer circle above. The player must break down the bricks to cause the circles to drop, then collect the correct one to complete the level. Two power-ups are embedded in the bricks, the +100 power-up awards bonus score, and the bomb power-up destroys multiple surrounding bricks at once.

Collecting the wrong answer circle or dropping the correct one results in losing a life, as does running out of time. The ball respawns on the paddle if it falls off the bottom. The same scoring and multiplier system applies throughout.

Level Design

Like Space Shooter, Brick Breaker's arena is constructed directly in the Canvas draw function. The brick layout is defined through fixed constants, four columns of five rows. As shown in Figure 5.30, each brick is rendered using the user's chosen theme colours, alternating between the two across the wall. A 3D shading effect is also applied to each brick, with a highlight along the edges and a dark shadow, giving it that retro look.

```
ctx.strokeStyle = COLORS.wall;
ctx.lineWidth = 3;
ctx.lineCap = "round";
ctx.strokeRect(4, 4, CANVAS_WIDTH - 8, CANVAS_HEIGHT - 8);

bricksRef.current.forEach(brick => {
  if (brick.alive) {
    ctx.fillStyle = brick.color;
    ctx.fillRect(brick.x, brick.y, BRICK_WIDTH, BRICK_HEIGHT);

    ctx.strokeStyle = COLORS.background;
    ctx.lineWidth = 3;
    ctx.strokeRect(brick.x, brick.y, BRICK_WIDTH, BRICK_HEIGHT);

    ctx.fillStyle = "rgba(255, 255, 255, 0.3)";
    ctx.fillRect(brick.x + 3, brick.y + 3, BRICK_WIDTH - 6, 3);
    ctx.fillRect(brick.x + 3, brick.y + 3, 3, BRICK_HEIGHT - 6);

    ctx.fillStyle = "rgba(0, 0, 0, 0.3)";
    ctx.fillRect(brick.x + 3, brick.y + BRICK_HEIGHT - 6, BRICK_WIDTH - 6, 3);
    ctx.fillRect(brick.x + BRICK_WIDTH - 6, brick.y + 3, 3, BRICK_HEIGHT - 6);
  }
});
```

Figure 5.30 – Brick Breaker, level design

Ball Physics

The most interesting aspect of Brick Breaker to implement was the ball physics. Rather than bouncing at fixed angles, the ball's horizontal direction is influenced by where it strikes the paddle, giving the player meaningful control over the ball's trajectory and allowing them to aim at specific columns. Figure 5.31 shows the angle calculation logic.

```

const paddle = paddleRef.current;
if (
  ball.y + BALL_RADIUS >= paddle.y &&
  ball.y - BALL_RADIUS <= paddle.y + PADDLE_HEIGHT &&
  ball.x >= paddle.x &&
  ball.x <= paddle.x + PADDLE_WIDTH
) {
  ball.dy = -Math.abs(ball.dy) * 1.15;
  const hitPos = (ball.x - paddle.x) / PADDLE_WIDTH;
  ball.dx = (hitPos - 0.5) * ball.speed * 2.5;
  if (Math.abs(ball.dx) < 1.0) {
    ball.dx = (Math.random() > 0.5 ? 1.0 : -1.0) * ball.speed;
  }
}

if (ball.y - BALL_RADIUS > CANVAS_HEIGHT) {
  ball.active = false;
}
});

```

Figure 5.31 – Brick Breaker, level design

Visual Feedback

Brick Breaker carries over the same feedback system from the previous two games. Each brick broken triggers a visual break effect and sound as shown in Figure 5.32.

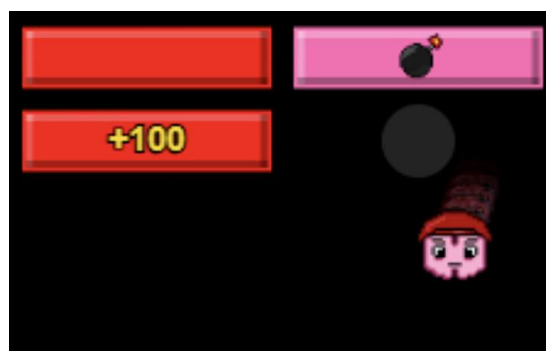


Figure 5.32 – Brick Breaker, brick visual feedback

Bomb bricks trigger an explosion clearing surrounding bricks as shown in Figure 5.33.



Figure 5.33 – Brick Breaker, bomb visual feedback

Furthermore, correct and incorrect answer collection triggers the same green and red effects used in the other games, keeping it consistent throughout the platform.

Shared Game UI

Each game shares a consistent set of screens ensuring a familiar experience across the platform.

The Level Complete screen shown in Figure 5.34 appears briefly between levels displaying the current score and multiplier before the next level begins.



Figure 5.34 – Level Complete overlay

The You Died screen shown in Figure 5.35 informs the player of their remaining lives and confirms the multiplier has reset to 1.0x.

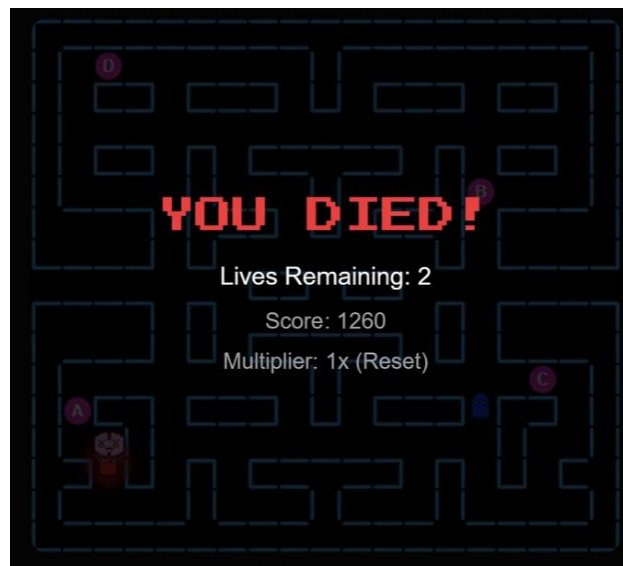


Figure 5.35 – You Died overlay

The You Win screen shown in Figure 5.36 displays the final score alongside the 500 point completion bonus.

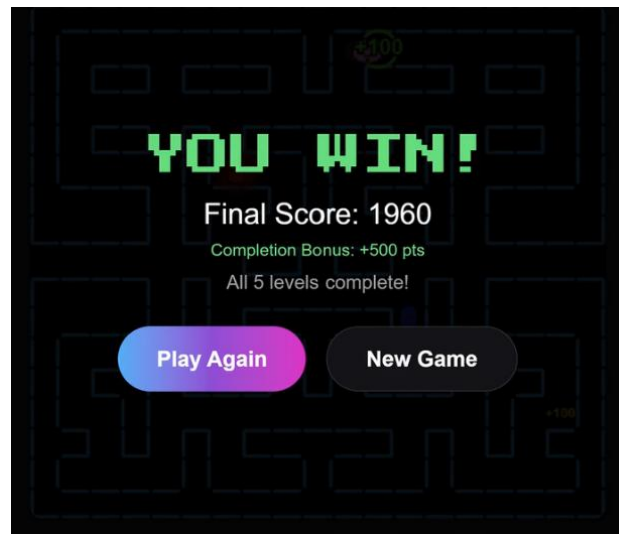


Figure 5.36 – You Win overlay

The Game Over screen shown in Figure 5.37 displays the final score and level reached with Play Again and New Game options.

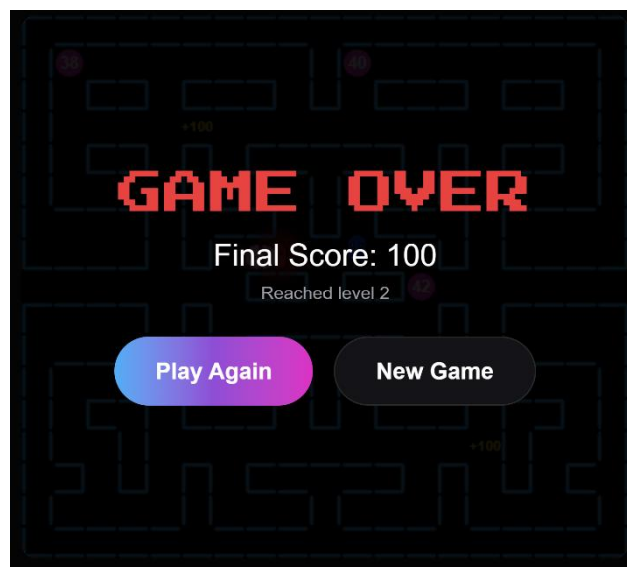


Figure 5.37 – Game Over overlay

5.4 Platform Features

The following section outlines the implementation of the key platform features.

Game Creation Flow

The game creation flow is the core user journey of LearnCode, guiding users through three steps to create a fully customised educational arcade game. As shown in Figure 5.38, the first step presents the three game templates as cards, with the Next button disabled until a selection is made.

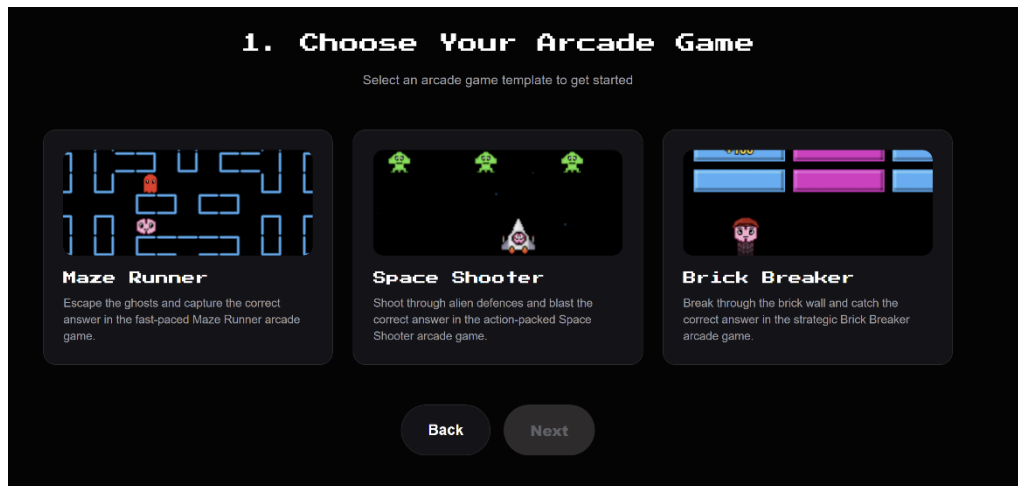


Figure 5.38 – Game creation flow, Game selection

Figure 5.39 shows the second step where the user selects their educational topic.

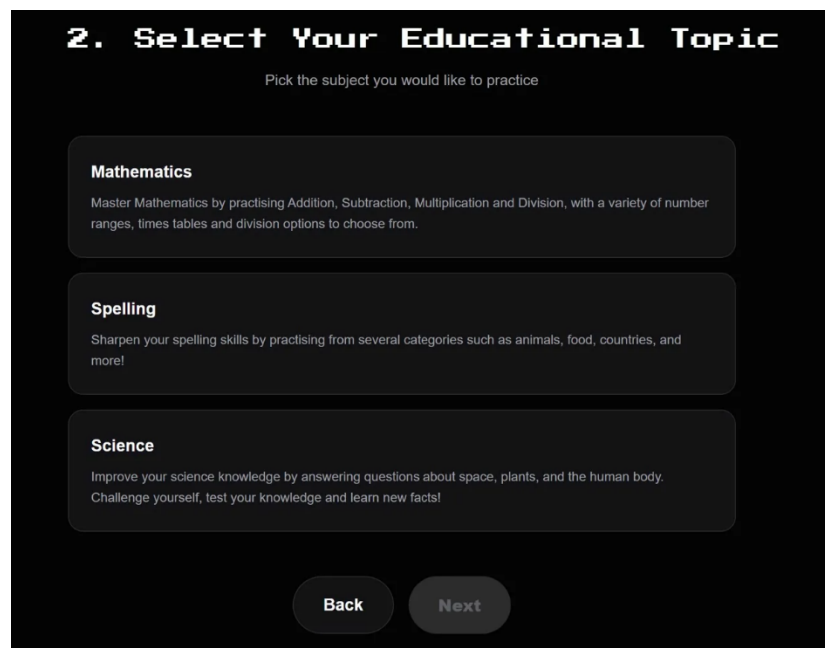


Figure 5.39 – Game creation flow, topic selection

The third step, shown in Figure 5.40, prompts the user to enter game details, Mathematics presents an operation dropdown, while Spelling and Science reveal a category dropdown.

3. Enter Details About Your Game
Customize your arcade game to match your needs and preferences

Operation
Select an operation

Lives
Select lives

Number of Questions
Select number of questions

Time Per Question
Less time = Higher base score per level
Select time per question

Custom Theme (Optional)
Enter two colours to customize your game theme. If left empty, a default theme will be used.
e.g. red and pink, blue and green etc.

Back Create Game

Figure 5.40 – Game creation flow, game details

All topics share three common options, number of lives, number of questions and time per question, where a shorter timer awards a higher base score per level as a difficulty option, as shown in Figure 5.41:

Time Per Question
Less time = Higher base score per level

Select time per question

Select time per question

40 seconds - EASY (100 base score)

30 seconds - MEDIUM (200 base score)

20 seconds - HARD (300 base score)

Figure 5.41 – Game creation flow, timer

Once Create Game is pressed, a loading screen is displayed as shown in Figure 5.42 while the API call is made.



Figure 5.42 – Game creation flow, loading screen

Game Sharing

When a game is generated, the full configuration is saved to Supabase alongside a unique six-character share code. To load a shared game, users must enter the code into the Game Share Code modal shown in Figure 5.43, which searches the database for a match and loads the exact same questions and configuration. The user must be logged in to share game codes.

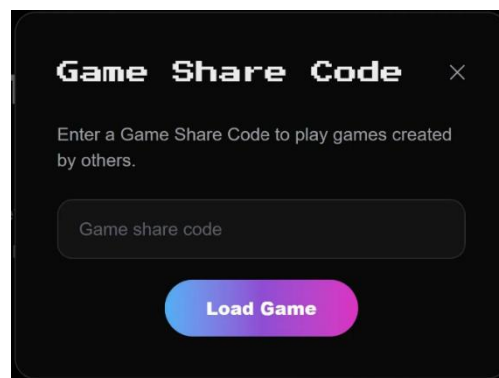


Figure 5.43 – Game sharing, game share code modal

Account Profile

User authentication is handled through Supabase Auth [31]. As shown in Figure 5.44, users can create an account or log in through modal accessed through a navbar link.

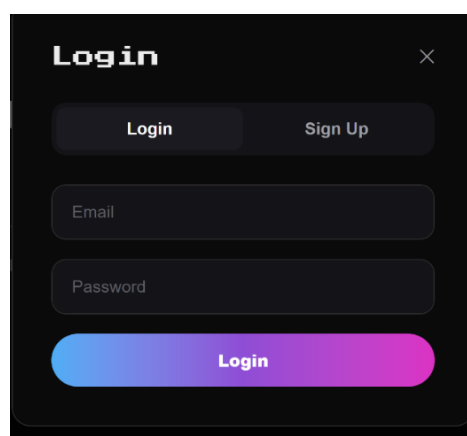


Figure 5.44 – Account profile, login/sign up modal

Figure 5.45 shows the profile page displaying the user's stats, Score Earned filter, Best Game and Recent Games sections.

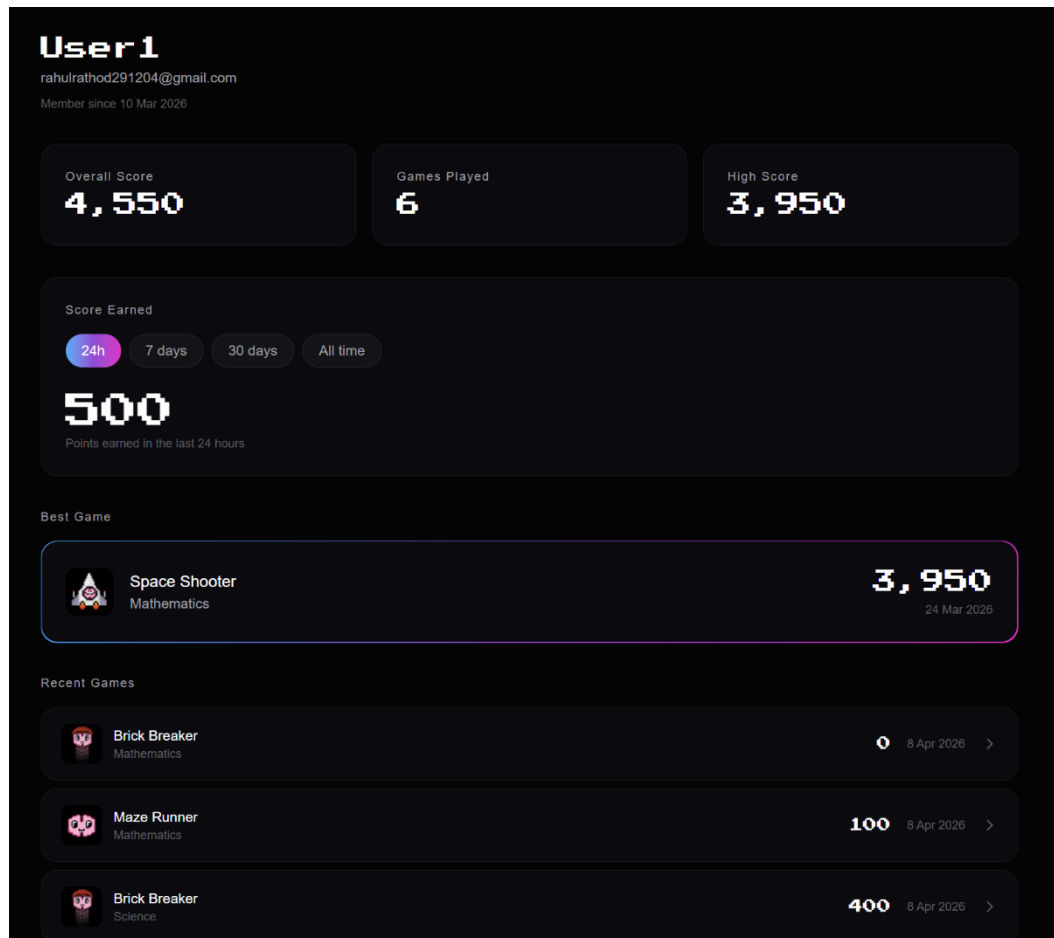


Figure 5.45 – Account profile, profile page

Clicking a recent game opens a modal shown in Figure 5.46 with the full game configuration, share code and a Play Again button.

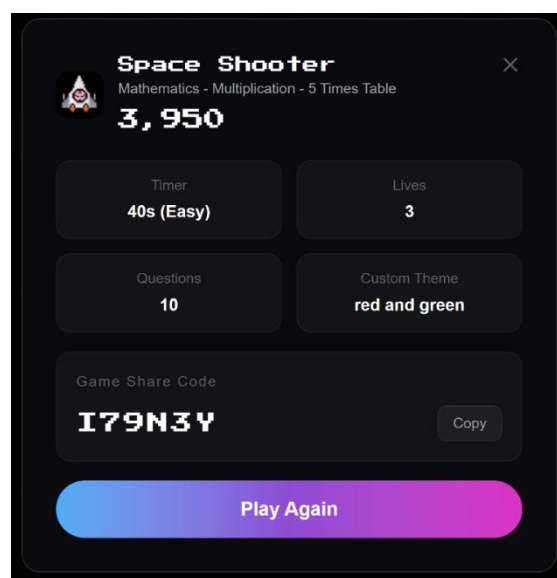


Figure 5.46 – Account profile, game overview modal

Classroom Mode

Classroom Mode allows teachers to host live game sessions with a real-time leaderboard. As shown in Figure 5.47, the navbar modal presents two options, Host and Join.

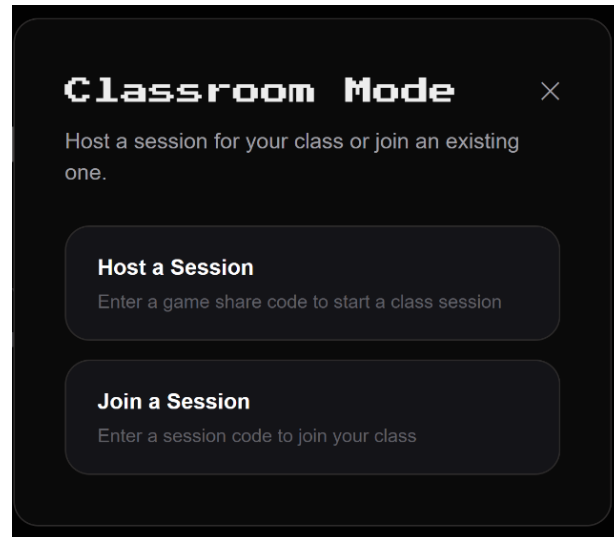


Figure 5.47 – Classroom mode, host or join modal

The host option allows the user to enter a share code and session duration as shown in Figure 5.48

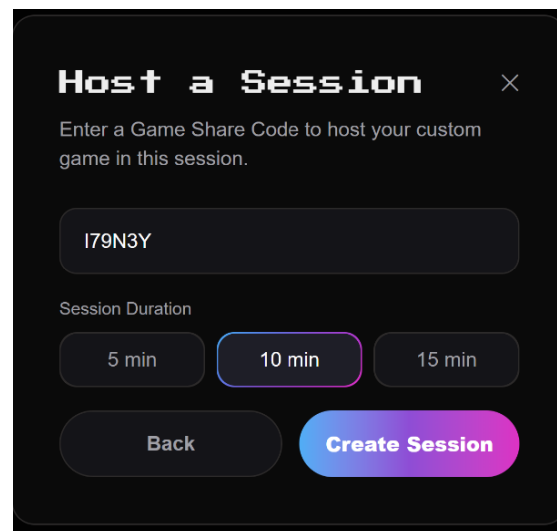


Figure 5.48 – Classroom mode, host a session modal

Once the host creates a session it directs them to the Waiting Room shown in Figure 5.49.

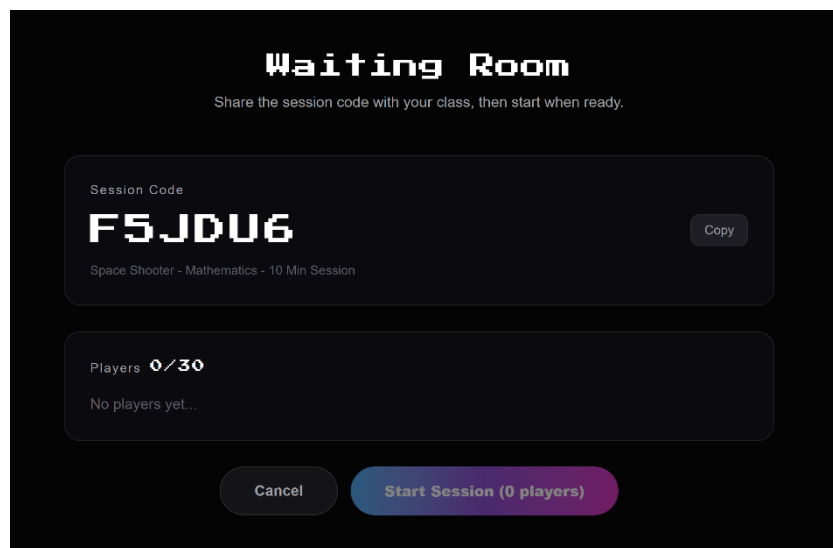


Figure 5.49 – Classroom mode, waiting page

On the other hand, the join option allows users to enter a session code provided by the host. Users can enter using their account or a guest nickname as shown in Figure 5.50.

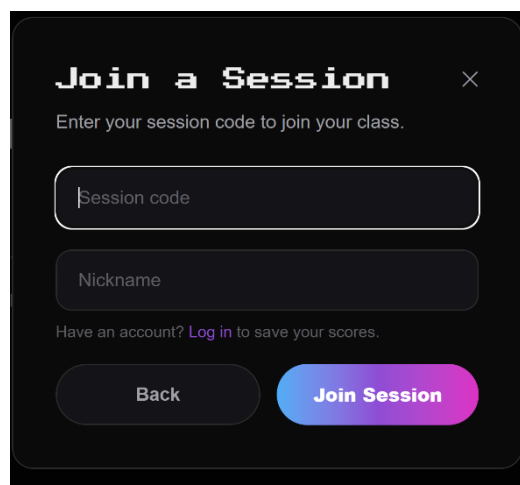


Figure 5.50 – Classroom mode, Join session modal

Once started, the joiners start playing, and the host is directed to the leaderboard page. The leaderboard updates in real time through Supabase Realtime [32] as shown in Figure 5.51.

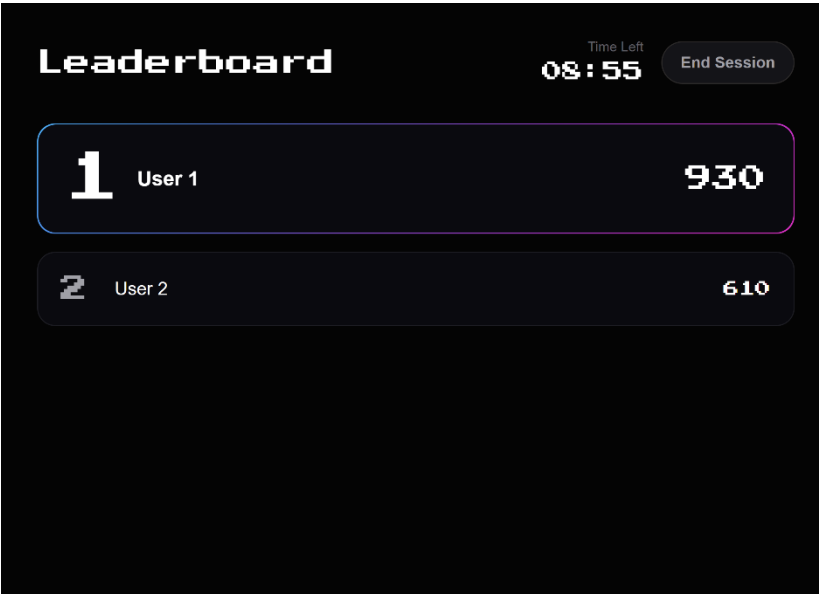


Figure 5.51 – Classroom, live leaderboard

5.5 Database

LearnCade uses Supabase across four tables, games, scores, sessions and session_players, each responsible for a different area of the platform. Figure 5.52 shows the four tables as they appear in the Supabase dashboard.

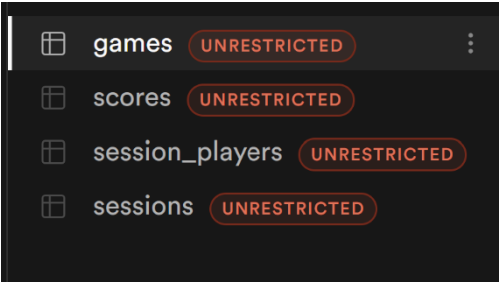


Figure 5.52 – Database, Supabase table dashboard

The games table stores every game's full configuration and assigns each entry a unique six-character share code, as shown in Figure 5.53.

game_type text	subject text	difficulty text	share_code text
space-shooter	mathematics	easy	EZX0R1
maze-runner	mathematics	easy	X5L98Y
maze-runner	mathematics	easy	RJB99I
space-shooter	spelling	easy	J1PPZ7
space-shooter	mathematics	easy	I79N3Y

Figure 5.53 – Database, games table

The scores table tracks each score earned by a user, linked to their account, allowing the Account Profile to calculate overall score and high score, as shown in Figure 5.54.

game_id	uuid	score	int4
fc563d5f-2aa8-4ae7-b939-8b0f0...	→	100	
ac0b2f9c-7203-48fb-95f2-22230...	→	3950	
ac9333e8-324a-4343-8c2a-d309...	→	0	
6c1bbaa7-064e-4e81-8ff9-f15e49...	→	100	
f0b2e67c-f7db-4b14-9d5e-20ea3...	→	400	
c7085555-91ec-4109-a947-d5e04...	→	0	

Figure 5.54 – Database, scores table

The sessions and session_players tables together power Classroom Mode. Figure 5.55 shows the sessions table storing each session's code, status and duration.

session_code	text	status	text	duration_minutes	int4
RJ58L3		ended		10	
F5JDU6		active		10	
68QH0N		ended		10	

Figure 5.55 – Database, session table

Figure 5.56 shows the session_players table tracking each participant's nickname and real-time score, which drives the live leaderboard.

session_id	uuid	nickname	text	score	int4
8443ba8b-e882-4f42-86bf-a1b22...	→	User 1		0	
3d0452df-13f4-4d1d-aa26-25ef25...	→	User 1		930	
ffd7b1c5-c62e-47f8-a0a9-cabfae7...	→	user 1		0	
3d0452df-13f4-4d1d-aa26-25ef25...	→	User 2		610	
02d3428b-4646-40ab-ad60-8c99...	→	User 1		0	

Figure 5.56 – Database, session_players table

5.6 Landing Page

The finished landing page closely reflects the wireframe layout presented in Section 4.4. The hero section shown in Figure 5.57 features the UVP tagline and a video showcase, with a prominent Start Creating call to action directing users to the game creation flow.

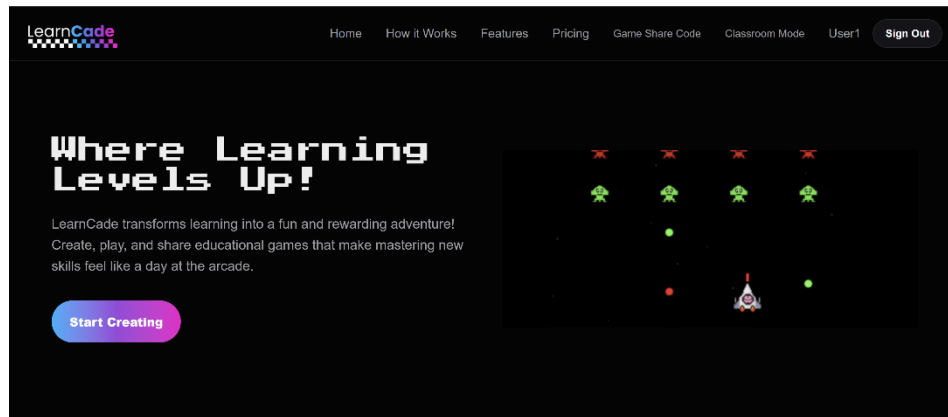


Figure 5.57 – Landing page, hero

A How it Works section shown in Figure 5.58 walks users through the core steps of the platform, there are also steps three and four that are not shown in this example.

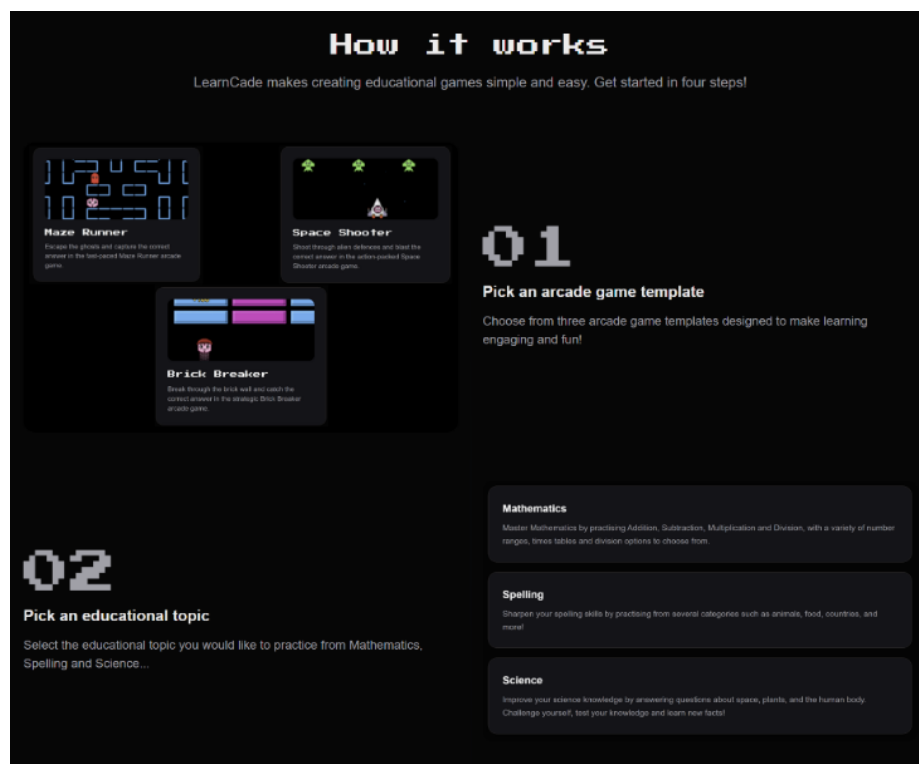


Figure 5.58 – Landing page, how it works

A Features section shown in Figure 5.59 highlights Game Sharing, Account Profile and Classroom Mode, with cards highlighted on hover using the brand gradient.

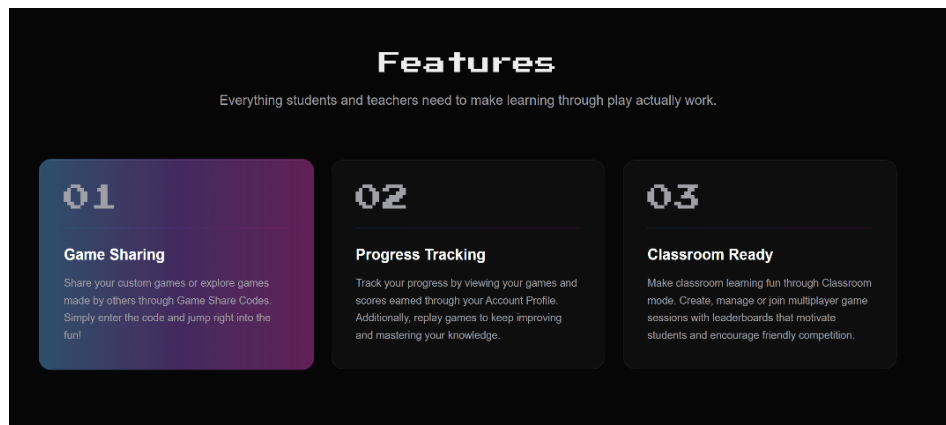


Figure 5.59 – Landing page, features

Finally, a Pricing section shown in Figure 5.60 presents the Free and Premium plans side by side, with a footer displaying the logo and copyright information below.

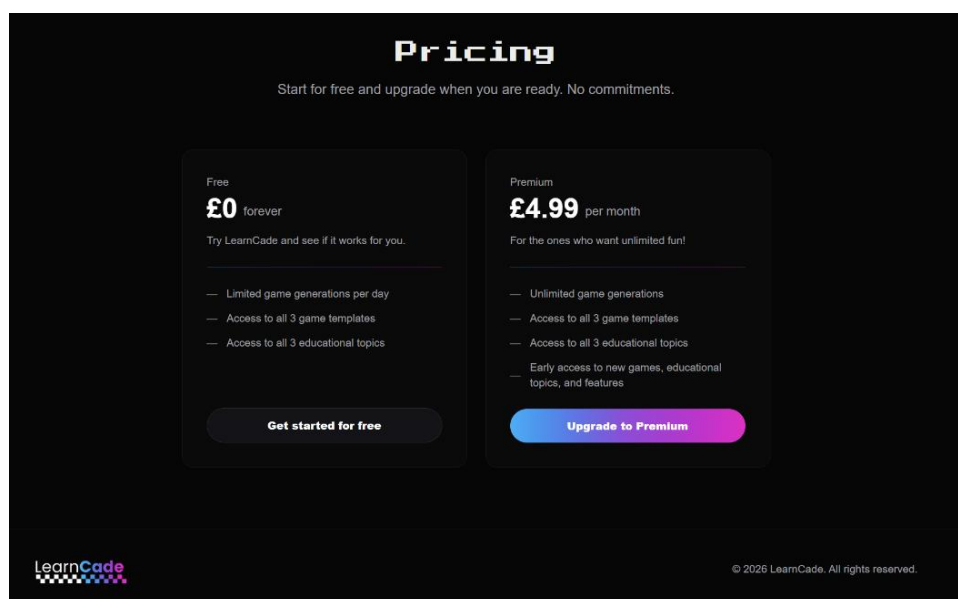


Figure 5.60 – Landing page, pricing and footer

5.7 Play Page

The play page brings together the arcade games, educational content and surrounding UI into a single cohesive interface, built directly from the wireframe designs in Section 4.4.

As shown in Figure 5.61, the game canvas occupies the centre, with the controls panel to the left and game configuration to the right.

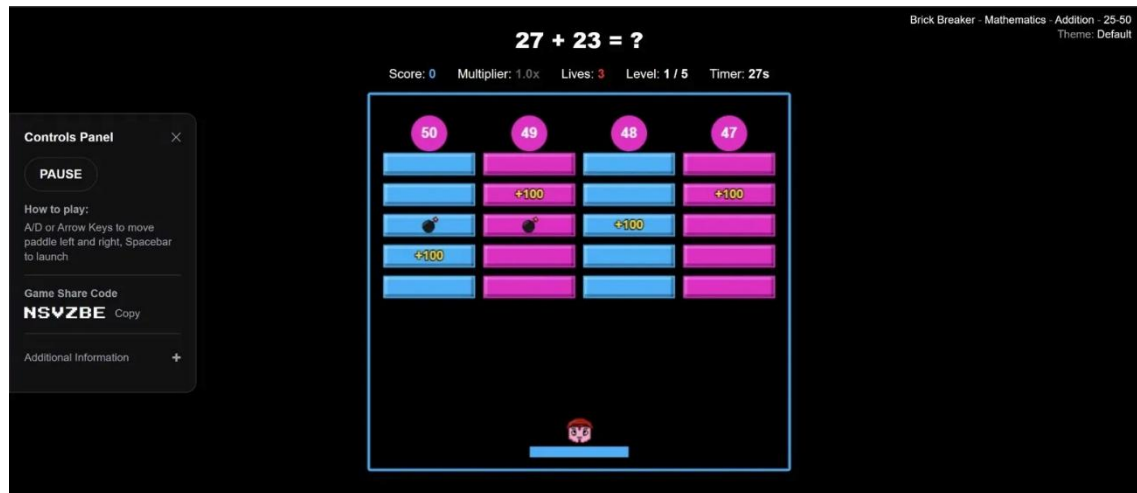


Figure 5.61 – Play page

The controls panel contains the Pause button, a How to Play reminder, the Game Share Code with a Copy button, and an Additional Information toggle. Pressing Pause overlays a PAUSED screen on the canvas as shown in Figure 5.62.

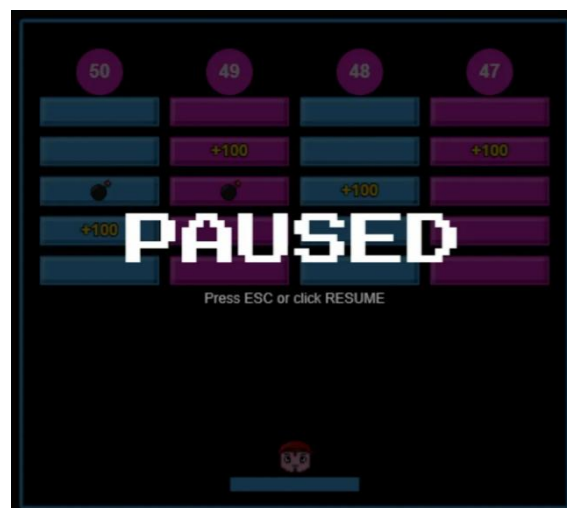


Figure 5.62 – Play page, paused overlay

Figure 5.63 shows when a user expands the Additional Information section. It reveals power-up descriptions for the active game, which can be useful for first time players.

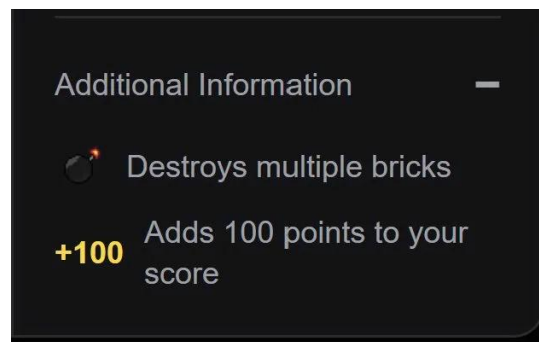


Figure 5.63 – Play page, additional information

When a player joins a Classroom Mode session, the top right panel also displays their nickname, confirming who they are competing as. This can be seen in Figure 5.64.

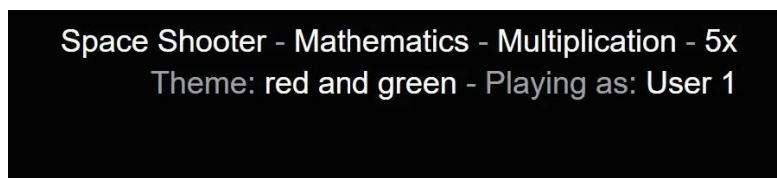


Figure 5.64 – Play page, full game configuration

If the viewport drops below the minimum required width, a full screen overlay pauses the active game and prompts the user to use full screen, as shown in Figure 5.65, ensuring the game is always played at the correct dimensions.

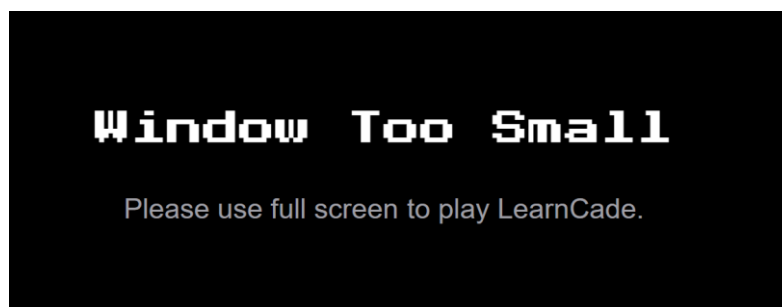


Figure 5.65 – Play page, viewport detection

5.8 Error Handling

Informative error handling was implemented across LearnCade's key interactions, fulfilling the approach outlined in Section 4.6.

As shown in Figure 5.66, invalid login credentials display an error message directly below the input fields.

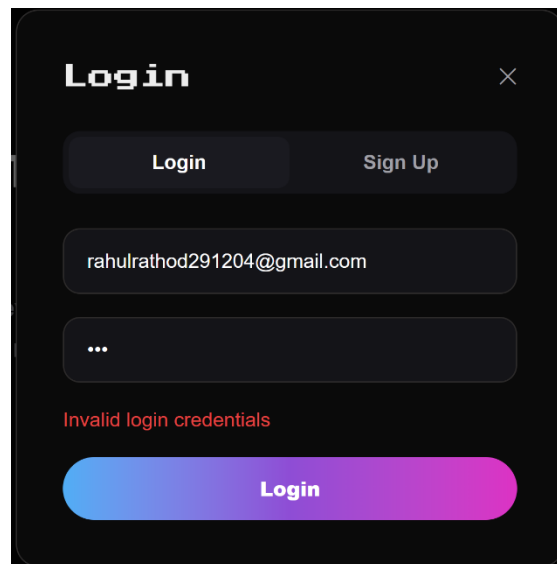


Figure 5.66 – Error handling, login

As shown in Figure 5.67, an invalid share code also displays a clear error message in the modal.

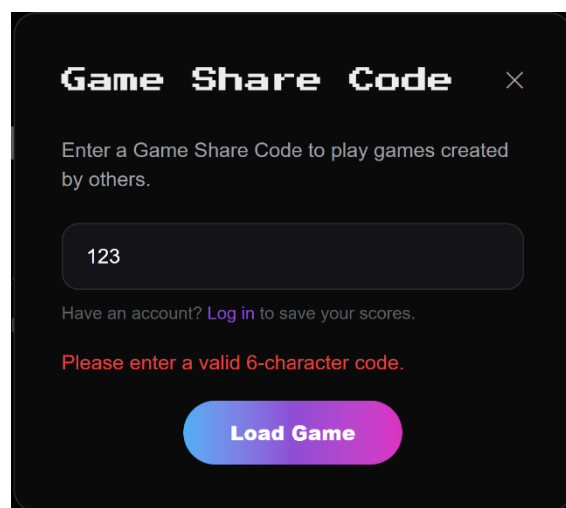
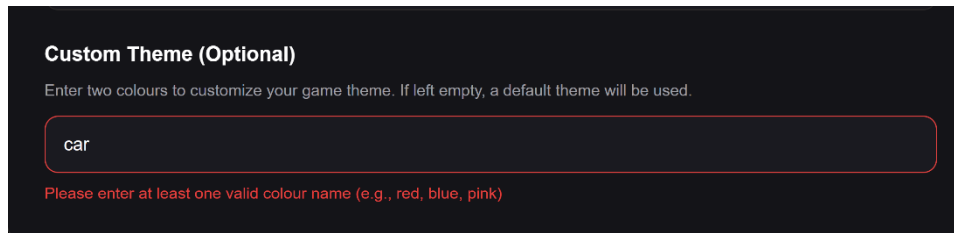


Figure 5.67 – Error handling, game sharing

Additionally, Figure 5.68 shows that an invalid custom theme entry will highlight the input in red, display an error message and also disable the Create Game button until corrected.



Custom Theme (Optional)
Enter two colours to customize your game theme. If left empty, a default theme will be used.

car

Please enter at least one valid colour name (e.g., red, blue, pink)

Figure 5.68 – Error handling, custom theme

As shown in Figure 5.69, API generation failures trigger a fallback screen displaying the error alongside a Create a Game button, ensuring users are never left on a broken screen.

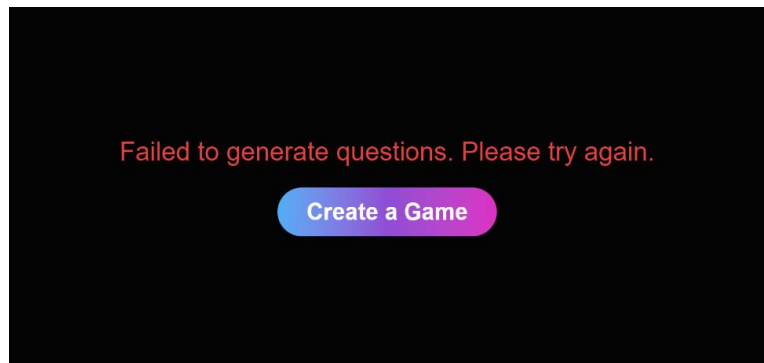


Figure 5.69 – Error handling, game generation

5.9 Deployment

LearnCade has been successfully deployed to Vercel, making the platform publicly accessible through a live and secure URL. The API key and Supabase credentials are securely configured through Vercel's environment variable system, ensuring sensitive keys are never exposed to the client.

6. Results and Discussion

The following chapter discusses the success of LearnCade against the aims and objectives stated at the start of the dissertation.

6.1 Aims Evaluation

This section evaluates whether LearnCade has met the aims stated in Section 1.1.

Achieved: Aim 1 – Balance education and entertainment

This aim has been achieved as all three games ensured that educational value was delivered through the questions and answers embedded in the gameplay rather than feeling like a separate test. The inclusion of arcade style power ups and multipliers maintained excitement while the educational questions provided meaningful learning.

Achieved: Aim 2 – Deliver a strong arcade experience

This aim was achieved as all three games delivered a strong arcade experience through pixel art visuals, retro sound effects, dark backgrounds, vibrant colours and arcade-style gameplay. All three games were also based on popular classic arcade titles, ensuring the platform feels familiar and engaging for users from the start.

Achieved: Aim 3 – Transform play-based learning

This aim was achieved as LearnCade automatically generates structured questions, answers and custom themes based on the user's selections. This means every game session is instant, unique and personalised, which is exactly what transforms LearnCade from traditional play-based learning platforms into a genuinely adaptive learning experience.

Achieved: Aim 4 – Support classroom learning through Classroom Mode

Classroom Mode supports learning by creating a fun, competitive environment where students play together and stay engaged. The shared experience encourages participation, increases motivation, and makes lessons feel more interactive compared to traditional paperwork.

Achieved: Additional Aims – Game Sharing and Account Profiles

The additional aims were also both achieved. Game sharing allows users to easily share the games they create, and the profile system provides players with access to their recent games and key stats, encouraging engagement and giving users a sense of progression.

6.2 Objectives Evaluation

This section evaluates whether LearnCade has met the objectives stated in Section 1.2:

Achieved: Objective 1 – Game Creation Flow

LearnCade includes a fully functional game creation flow allowing users to select a template, topic and configure parameters such as questions, lives and a timer. The optional custom theme feature dynamically applies user-chosen colours to the game environment.

A future improvement would be adding a year group filter to tailor content to different ability levels.

Achieved: Objective 2 – Develop Arcade Games

Three fully functional arcade games and their assets were developed, each including movement mechanics, collision detection, scoring systems, multipliers, power ups and seamless API integration.

A future improvement would be adaptive difficulty based on player performance.

Achieved: Objective 3 – Integrate the API

The Claude API was successfully integrated to generate questions, answers and custom themes, with prompt engineering ensuring consistent outputs across all subjects. API failures are handled gracefully with clear error messages.

One limitation is that negative numbers are not currently supported in Mathematics.

Achieved: Objective 4 – Strong UI/UX Design

LearnCade features a polished interface with eye-catching visuals, consistent styling and clear navigation, successfully balancing the needs of both children and teachers. Detailed wireframes played a key role in refining the layout before implementation.

Future improvements could include stronger accessibility features such as screen reader support.

Achieved: Objective 5 – Create Classroom Mode

Classroom Mode was successfully implemented, with Supabase's real-time capabilities enabling an intuitive host, join flow and reliable leaderboard updates.

A future improvement would be adding detailed analytics such as individual question performance.

Achieved: Objective 6 – Enable Game Sharing

Each game creation generates a unique share code, allowing users to share identical game experiences across different devices.

A future improvement would be adding QR code generation to make sharing even easier.

Achieved: Objective 7 – Account Profile functionality

Users can create accounts and access a profile displaying their stats, recent games and share codes.

Future enhancements could include profile avatars and dedicated teacher accounts.

Achieved: Objective 8 – Database Integration

Supabase was used to store user accounts, saved games, scores and leaderboard data reliably.

A future improvement would be implementing stronger database security for large scale deployment.

6.3 System Testing

To evaluate LearnCade, a heuristic evaluation was conducted using Nielsen's 10 usability heuristics [33]. This method allowed me to find usability issues without requiring formal testing with real users. Figure 6.1 presents the full results.

Heuristic Evaluation

Evaluator Name: Rahul Rathod

Device / Browser/ OS: AsusZenBook/ Chrome / Windows

App / Version: LearnCade/v1.0

SEVERITY RATING

0 = I don't agree that this is a usability problem at all

1 = Cosmetic problem only: fix if time is available

2 = Minor usability problem: fixing this should be given low priority

3 = Major usability problem: important to fix, given high priority

4 = Usability catastrophe: fix this before product can be released

HEURISTICS	VIOLATION	RECOMMENDATION	SEVERITY
1. Visibility of system status <i>Always keep users informed about what is going on, through appropriate feedback within reasonable time.</i>	Score, lives, timer and multiplier are always visible during gameplay. A loading screen is displayed during game generation and a overlay appears when the game is paused. No violation.	None required.	0
2. Match between system and the real world <i>Follow real-world conventions, making information appear in a natural and logical order.</i>	Familiar arcade and gameplay mechanics are used throughout. Educational topics and game options are language and age-appropriate. No violation.	None required.	0
3. User control and freedom <i>Users should leave the unwanted state without having to go through an extended dialogue, undo and redo.</i>	Users can pause at any time, return to the creation flow and replay previous games from their profile. Back and Next buttons are present throughout the creation flow. Violation: Once a game is generated, users cannot edit parameters without restarting the creation flow entirely.	Allow users to edit game parameters after generation without losing their progress.	2
4. Consistency and standards <i>Users should not have to wonder whether different words, situations, or actions mean the same thing.</i>	A consistent navbar, colour scheme, pixel font and button styles are applied across all pages. All three games share the same scoring system, multiplier and UI conventions. No violation.	None required.	0
5. Error prevention <i>Either eliminate error-prone conditions or check for them and present users with a confirmation option before they commit to the action.</i>	Strong error handling across all key interactions. The Create Game button is disabled until all inputs are valid and the Next button is disabled until a selection is made at each creation step. Violation: The custom theme is a text entry box, users could enter colours that the API may not understand.	Implement a dropdown or colour picker for the custom theme option.	2

HEURISTICS	VIOLATION	RECOMMENDATION	SEVERITY
6. Recognition rather than recall <i>Minimize the user's memory load by making objects, actions, and options visible.</i>	Game templates and topics are displayed as clearly labelled cards. The controls panel and How to Play section are always accessible during gameplay without requiring the user to memorise controls. No violation.	None required.	0
7. Flexibility and efficiency of use <i>Accelerators. Allow users to tailor frequent actions.</i>	The Play Again button allows quick replay without going through the creation flow. The profile page allows users to replay any previously created game instantly. Violation: No keyboard shortcuts exist outside of gameplay. Violation: Users must click through the full three step creation flow every time with no option to save preferences.	Add keyboard shortcuts Allow users to save favourite game configurations for quick replay.	2
8. Aesthetic and minimalist design <i>Dialogues should not contain information which is irrelevant or rarely needed.</i>	The dark arcade aesthetic maintains a clear visual hierarchy. UI elements are minimal and focused on the game experience. The controls panel can be collapsed to further reduce visual distraction during gameplay. No violation.	None required.	0
9. Help users recognize, diagnose, and recover from errors <i>Error messages should be expressed in plain language (no codes), precisely indicate the problem, and constructively suggest a solution.</i>	Informative error messages are displayed across all key interactions. API generation failures trigger a dedicated fallback screen with a Create a Game button redirecting users back to the creation flow. No violation.	None required.	0
10. Help and documentation <i>Even though it is better if the system can be used without documentation, it may be necessary to provide help and documentation.</i>	An objective and controls modal appears before each game begins. An Additional Information panel provides power-up descriptions during gameplay and is accessible at any time from the controls panel. Violation: The platform has no dedicated help page or FAQ section.	Add a dedicated help or FAQ section to the landing page for first time users.	3

Figure 6.1 – Heuristics Evaluation, template taken from Figma community [34]

The evaluation confirmed that LearnCade performs well across the majority of heuristics, particularly in areas of consistency, error handling and visual feedback, validating the design decisions made throughout development. The primary improvement identified is the addition of a dedicated help or FAQ section, which will be prioritised in future development.

7. Critical Appraisal

The following chapter provides a critical evaluation of LearnCade's development.

7.1 What Went Well

Overall, LearnCade developed into a platform that closely reflects the vision outlined at the start, with all aims and objectives met.

The Claude API integration was one of the strongest aspects, producing consistently structured and age-appropriate outputs across all subjects and games.

The three arcade games were another significant success. Building Maze Runner, Space Shooter and Brick Breaker was challenging, but the final results delivered a genuine arcade feel that clearly differentiated LearnCade from competitors.

Classroom Mode was also a strong achievement, with the host and join flow, session code system and waiting room all coming together to create a feature that could realistically be used in a school setting.

The design process was also well executed. Creating detailed wireframes before development began meant key layout decisions were made early, and the pixel font, dark background, brand gradient and consistent styling gave the platform a strong visual identity.

7.2 What Didn't Go As Well

Despite the overall success, there are areas that would be approached differently or improved in the future.

Responsive design was the most significant limitation. While most pages adapt across screen sizes through Tailwind CSS, the play page was designed around a fixed pixel HTML5 Canvas that does not scale across viewports. A window size detection system was implemented as mentioned in Section 5.7, however, the platform is not full responsive. Additionally, it lacks support for tablet and mobile devices. A dynamic canvas scaling approach will be implemented in future development.

Formal user testing was another limitation. While testing was conducted through a heuristics evaluation, as shown in Section 6.3, no actual user testing was carried out. Gathering feedback from teachers and students would have provided valuable insights. As discussed in Section 3.5.4, the planned headteacher meeting could be the opportunity to conduct this.

Accessibility was also not fully addressed. While the platform maintains strong usability considerations, outlined in Section 4.6, features such as screen reader support, keyboard navigation and adjustable font sizes were not implemented and will be prioritised in future development.

7.3 Social, Commercial and Economic Considerations

From a social point of view, LearnCade has the potential to make a positive contribution to primary school education, as research cited in Section 2.1 demonstrates that educational games improve student motivation and retention. The free tier ensures the platform is accessible to all schools regardless of budget, which is particularly important for schools in low-income areas.

However, deploying LearnCade in schools would raise significant legal and safeguarding considerations. Compliance with UK GDPR would be essential before public launch, and the platform would need to meet the additional requirements under the Age Appropriate Design Code [35] given that it targets children aged 5 to 10. Safeguarding measures such as restricting account creation to verified school email addresses or requiring parental consent for users under 13 would also need to be considered.

Environmentally, the platform's current carbon footprint is minimal, however scaling to a larger user base would increase both server load and API usage. This is worth considering in the long-term as LLM API calls are notably more energy intensive than standard web requests.

7.4 Personal Development

LearnCade has been the most complex yet rewarding project I have undertaken at university, requiring skills across game development, UI/UX design, software engineering and business development.

Developing three fully functional arcade games deepened my understanding of game development, from designing the assets to implementing the game logic.

Working with Claude API gave me practical experience in AI integration that feels directly relevant to where the software industry is heading.

Furthermore, designing the platform's user interface taught me a great deal about UI/UX, particularly wireframing, balancing aesthetics with usability and designing for a dual audience.

Together, these experiences have strengthened my skills in areas that I hope to build a career around in the near future.

8. Conclusion

In conclusion, LearnCade is a web-based platform that successfully transforms classic arcade games into engaging, personalised educational experiences for children aged 5 to 10. This dissertation has documented the project's full journey, from its initial concept through to its business development, design, implementation and evaluation, covering the key decisions made at every stage.

All eight objectives were met. The integration of the Claude API to dynamically generate educational content proved to be the most significant achievement, ensuring every game session is unique and personalised to the user's selections.

The three arcade games, Maze Runner, Space Shooter and Brick Breaker, each delivered a genuine arcade experience that differentiates LearnCade from existing competitors. Classroom Mode demonstrated that the platform has genuine potential to be used in a real school setting.

Looking ahead, LearnCade has a clear path for future development. Expanding the range of educational topics and game templates, implementing full responsive support across all screen sizes, and improving accessibility are all key priorities. With these improvements, LearnCade has the potential to become a genuinely competitive product in the UK EdTech market.

9. References

- [1] N. Peña-Miguel and M. Sedano Hoyuelos, “Educational Games for Learning,” *Universal Journal of Educational Research*, vol. 2, no. 3, pp. 230-238, 2014.
- [2] Vercel, “Vercel,” [Online]. Available: <https://vercel.com/>. [Accessed Mar 2026].
- [3] Next.js, “Learn Next.js,” [Online]. Available: <https://nextjs.org/learn>. [Accessed Mar 2026].
- [4] TypeScript, “TypeScript: JavaScript with Syntax for Types,” [Online]. Available: <https://www.typescriptlang.org/>. [Accessed Mar 2026].
- [5] Anthropic, “Claude API,” [Online]. Available: <https://claude.com/platform/api>. [Accessed Mar 2026].
- [6] Supabase, “Supabase Database Overview,” [Online]. Available: <https://supabase.com/docs/guides/database/overview>. [Accessed Mar 2026].
- [7] Educaplay, “Educaplay,” [Online]. Available: <https://www.educaplay.com/>. [Accessed Mar 2026].
- [8] Kahoot, “Kahoot!,” [Online]. Available: <https://kahoot.com/>. [Accessed Mar 2026].
- [9] Kahoot, “Kahoot Reaches 10 Billion,” 9 Nov 2023. [Online]. Available: <https://kahoot.com/blog/2023/11/09/kahoot-reaches-10-billion/>. [Accessed Mar 2026].
- [10] SimilarWeb, “SimilarWeb Website Analysis,” [Online]. Available: <https://www.similarweb.com/website/>. [Accessed Mar 2026].
- [11] Department for Education, “Schools, pupils and their characteristics,” 5 Jun 2025. [Online]. Available: <https://explore-education-statistics.service.gov.uk/find-statistics/school-pupils-and-their-characteristics/2024-25>. [Accessed Mar 2026].
- [12] MyEdSpace, “The Value of Play: Why Fun Matters in Education,” 5 May 2025. [Online]. Available: <https://myedspace.co.uk/blog/post/the-value-of-play-why-fun-matters-in-education>. [Accessed Mar 2026].

- [13] N. Maher, "Lean Canvas - 1-page business plan," Figma, [Online]. Available: <https://www.figma.com/community/file/1105182362858244264/lean-canvas-1-page-business-plan>. [Accessed Mar 2026].
- [14] J. Aston, E. Davies, M. Guijon, K. Lauderdale and D. Popov, "The Education Technology Market in England," 2022.
- [15] Kahoot, "Q4 2022 Quarterly Report," 2023.
- [16] Lucidchart, "Lucidchart," [Online]. Available: <https://www.lucidchart.com/pages>. [Accessed Mar 2026].
- [17] Kahoot, "The most engaged workforces use Kahoot! 360," [Online]. Available: <https://kahoot.com/business/>. [Accessed Mar 2026].
- [18] Kahoot, "Elevate engaging learning in your team, school or district with Kahoot! EDU," [Online]. Available: <https://kahoot.com/schools/edu-school-and-district/>. [Accessed Mar 2026].
- [19] Vercel, "Vercel Pricing," [Online]. Available: <https://vercel.com/pricing>. [Accessed Mar 2026].
- [20] Anthropic, "Claude API Pricing," [Online]. Available: <https://platform.claude.com/docs/en/about-claude/pricing>. [Accessed Mar 2026].
- [21] Supabase, "Supabase Pricing," [Online]. Available: <https://supabase.com/pricing>. [Accessed Mar 2026].
- [22] Hostinger, "Domain name pricing," [Online]. Available: <https://www.hostinger.com/pricing/domains>. [Accessed Mar 2026].
- [23] A. Tweddle, "Freemium business model examples: key pros and cons," TrueLayer, 8 Oct 2024. [Online]. Available: <https://truelayer.com/blog/business-growth/your-guide-to-freemium-business-models/>. [Accessed Mar 2026].
- [24] AppSamurai, "Microtransaction," [Online]. Available: <https://appsamurai.com/glossary/microtransaction/>. [Accessed Mar 2026].
- [25] Figma, "Figma," [Online]. Available: <https://www.figma.com/>. [Accessed Mar 2026].
- [26] Krita, "Krita," [Online]. Available: <https://krita.org/en/>. [Accessed Mar 2026].
- [27] Namco, "Pac-Man," 1980.
- [28] Taito, "Space Invaders," 1978.
- [29] Atari, "Breakout," 1976.
- [30] Google Fonts, "Press Start 2P," [Online]. Available: <https://fonts.google.com/specimen/Press+Start+2P>. [Accessed Apr 2026].

- [31] Supabase, “Supabase Auth,” [Online]. Available: <https://supabase.com/docs/guides/auth>. [Accessed Apr 2026].
- [32] Supabase, “Supabase Realtime,” [Online]. Available: <https://supabase.com/docs/guides/realtime>. [Accessed Apr 2026].
- [33] J. Nielsen, “10 Usability Heuristics for User Interface Design,” Nielsen Norman Group, 24 Apr 1994. [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>. [Accessed Apr 2026].
- [34] Kapil, “Heuristic Evaluation Template,” Figma, [Online]. Available: <https://www.figma.com/community/file/905622673082476274/heuristic-evaluation-template>. [Accessed Apr 2026].
- [35] ICO, “Children's code guidance and resources,” [Online]. Available: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/childrens-code-guidance-and-resources/>. [Accessed Apr 2026].